

KLARITY

Cahier des charges technique et fonctionnel

Plateforme éducative IA — 3ème, 1ère, Terminale (Cameroun)

STACK — Next.js 15.5 · Prisma · PostgreSQL · Redis/BullMQ · Docker

PAIEMENT — NotchPay (Mobile Money uniquement) — implémenté, tests sandbox à venir

INTELLIGENCE ARTIFICIELLE — API Anthropic Claude — Haiku (chat/quiz) & Sonnet (correction/lacunes) — accès API en attente

VERSION — v1.32 — 10 septembre 2026

Document de référence pour l'équipe de développement et l'agent Claude Code. Consolide : spécifications fonctionnelles, sécurité, scalabilité et architecture d'implémentation.

Journal des modifications

v1.0 → v1.1

- §1.2 / §2.1 — Périmètre matières du MVP explicité : Maths, Physique, Français, Philosophie (+ Système d'Information, Programmation, Réseau pour la filière TI). Anglais, Histoire-Géo, SVT, Chimie sont hors scope MVP, exclusion volontaire et non un oubli.
- §2.1 — Chat-tuteur IA reformulé : le chat est désormais contextualisé à l'épreuve en cours de traitement (et non plus seulement à la matière), et strictement cadré au contenu scolaire.
- §2.1 — Correction automatisée clarifiée : la correction produite à la première tentative fait foi ; les tentatives suivantes (retraitement) réaffichent cette même correction sans nouvel appel au modèle IA.
- §2.4 — Référence corrigée : "voir 7.4" → "voir 6.4" (plafond technique du plan premium).
- §2.4 / §2.6 (nouveau) — Parcours de paiement explicité : le parent est prioritaire sur le paiement de l'abonnement, mais l'élève peut également payer lui-même. Page de paiement accessible depuis les deux espaces.
- §2.7 (nouveau) — Authentification élève précisée : connexion par code élève (ELE-XXX-XXX) + PIN à 4 chiffres, avec réinitialisation via le parent.
- §4.1 — Entité Eleve enrichie des champs d'authentification par PIN.
- §4.3 — Relation CorrectionDetail corrigée pour refléter une correction unique par épreuve/élève, non une correction par tentative.
- §4.4 — Entité ConversationChat enrichie d'un lien vers l'épreuve traitée.
- §4.5 — Entité Paiement enrichie du champ payeurRole.
- §5.5 — Rappel de renouvellement : notification envoyée aux deux canaux disponibles (parent si lien vérifié + élève) plutôt qu'à un seul destinataire.
- §6.1 / §6.4 — Note ajoutée sur l'absence de ré-appel Sonnet lors des tentatives de retraitement (cf. §2.1).
- §7 — Nouvelle ligne "Chat IA" dans la synthèse sécurité (cadrage scolaire strict, journalisation des dérives).

v1.1 → v1.2

- §1.2 / §2.1 / §2.2 — Inscription élève confirmée comme autonome : l'élève crée son compte seul, sans validation parentale préalable ; le lien parent ↔ élève se fait a posteriori (choix assumé, explicité dans les contraintes structurantes).
- §2.1 / §2.8 (nouveau) — Mécanisme de signalement d'une correction contestée introduit : bouton de signalement côté élève, file de traitement et override manuel côté admin.
- §2.3 — Back-office admin enrichi d'une liste des corrections signalées à traiter.
- §4.3 — Entité CorrectionDetail enrichie des champs de signalement et d'override admin.
- §4.7 — Nouvel index sur les corrections signalées non traitées.
- §7 — Précision confirmée : Klarity reste 100% XAF/CamerPay, aucune gestion diaspora/multi-devises (contrairement à d'autres projets) — choix assumé, pas un oubli.

v1.2 → v1.3

- §1.1 / §1.2 / §3 / §5.5 / §9 / §10 — Mode allégé WhatsApp retiré du périmètre MVP et repositionné en Phase 7 (post-MVP) : effort de développement jugé trop conséquent pour être mené en parallèle du cœur du produit (chaque flux — upload, quiz, chat contextualisé, correction — devrait être repensé pour une interface conversationnelle). La contrainte de connectivité limitée reste couverte au MVP par le design mobile-first de la version web.
- §4.3 — SessionActivite.canal conserve la valeur WHATSAPP dans l'enum pour éviter une migration de schéma ultérieure, mais seule WEB est produite au MVP.

v1.3 → v1.4

- §2.1 / §4.4 / §6.2 — Ambiguïté levée : le chat par matière (sans épreuve précise) coexiste bien avec le chat contextualisé à une épreuve — ce sont deux modes distincts et valides, tous deux strictement cadrés au contenu scolaire. ConversationChat.matiereId redevenu obligatoire (présent dans les deux modes) ; epreuveld reste le seul champ distinguant le mode actif. Interface AIProvider.chat() mise à jour en conséquence.

v1.4 → v1.5

- §2.1 / §4.3 — Détection de lacunes rendue immédiate : chaque point manqué identifié lors d'une correction crée ou met à jour directement une Lacune pour l'élève, sans attendre un pattern sur plusieurs tentatives. L'analyse croisée de l'historique garde un rôle allégé (affinage de niveauMaitrise, détermination de resolu).
- §2.1 / §2.5 — Vidéos de renforcement affichées directement sur l'écran de correction : conséquence de la création immédiate de Lacune, qui déclenche le pipeline vidéo automatisé existant sans nouveau mécanisme. Le quiz journalier reste inchangé (job cron quotidien, cf. §4.3).
- §4.3 — CorrectionDetail.pointsManques structuré en tableau d'objets {notion, detail}, avec notion partageant la même taxonomie que Lacune.notion, condition nécessaire pour la création automatique de Lacune et au matching du pipeline vidéo.
- §6.1 / §6.2 — AIProvider.detecterLacunes() renommée affinerLacunes(), rôle recentré sur l'affinage (niveauMaitrise/resolu) plutôt que la création des Lacune, désormais dérivée directement de la sortie de corrigerCopie().

v1.5 → v1.6

- §1.1 / §2.4 / §3 / §5 / §7 / §10 — Paiement par carte bancaire retiré du périmètre : Klarity ne propose désormais que le Mobile Money via CamerPay. Suite à la revue des maquettes, le formulaire carte (numéro, expiration, CVV) semblait s'afficher directement dans l'UI Klarity plutôt que dans une iframe CamerPay isolée — retirer entièrement l'option élimine ce risque de conformité PCI-DSS plutôt que de devoir garantir l'isolation stricte de ce flux.
- §4.5 — Paiement.methode réduit à la seule valeur MOBILE_MONEY, conservé en enum pour permettre une réintroduction future sans migration de schéma.

v1.6 → v1.7

- §1.2 / §2.1 — Chat-tuteur rendu généraliste : il couvre désormais l'ensemble du programme scolaire de la classe/filière de l'élève, y compris les matières sans banque d'épreuves (Anglais, Histoire-Géo, SVT, Chimie). La restriction MVP (Maths, Physique, Français, Philo, TI) ne s'applique plus qu'à la banque d'épreuves et à la correction automatisée.
- §4.2 — Entité Matière étendue à toutes les matières du programme, avec un nouveau champ banqueDisponible distinguant les matières avec épreuves/correction (true) de celles couvertes uniquement par le chat (false).
- §7 — Ligne "Chat IA" mise à jour : cadrage étendu au programme scolaire complet plutôt qu'aux seules matières avec banque d'épreuves.

v1.7 → v1.7.1

- §1.2 / §2.1 / §4.2 — Précision réintroduite (perdue par erreur lors de la réécriture v1.7 de l'entité Matière) : Système d'Information, Programmation et Réseau sont réservées exclusivement aux élèves de 1ère et Terminale ayant choisi la filière TI — la 3ème n'a pas de filière et n'y a donc jamais accès, pas plus qu'un élève de filière générale.

v1.7.1 → v1.8

- §1.2 / §2.1 / §4.2 — Socle de matières de la banque d'épreuves/correction rendu dépendant de la classe, pour coller au programme camerounais réel : Philosophie n'est plus proposée en 3ème (elle n'y est pas enseignée) — elle reste réservée à 1ère/Terminale. SVT est ajoutée à la banque d'épreuves et à la correction automatisée, exclusivement pour la 3ème, en remplacement de Philosophie sur cette classe.
- §4.2.2 — Par défaut, la correction SVT suit le principe des matières scientifiques (vérification par barème, pas de mécanisme ExempleCorrection few-shot) — à confirmer ou ajuster si SVT nécessite en réalité une évaluation plus méthodologique (schémas, rédaction structurée).

v1.8 → v1.9

- §2.4 — Précision tranchée : la correction automatisée par IA et la détection immédiate de lacunes (§2.1, §4.3) sont incluses dans le plan Gratuit, sur les 2 épreuves par matière autorisées — ce ne sont pas des fonctionnalités réservées au Premium. Seuls le volume (nombre d'épreuves/quiz) et les vidéos de renforcement (§2.5) restent des différenciateurs Gratuit/Premium.

v1.9 → v1.10

- §2.1 / §4.3 / §6.2 / §7 — Upload multi-pages confirmé : une copie d'examen s'étale souvent sur plusieurs pages de cahier. TentativeEpreuve.photoUploadKey (unique) devient photoUploadKeys (tableau ordonné de clés R2). Toutes les pages sont transmises ensemble à Claude Sonnet en une seule requête vision (pas d'appel IA par page). Validation MIME/taille/re-encodage désormais appliquée individuellement à chaque page.

v1.10 → v1.11

- §2.2 — Export PDF du rapport mensuel parent ajouté : génération à la demande à partir des données existantes, pas de stockage persistant.
- §2.2.2 (nouveau) — Alertes intelligentes côté parent : règles CRITIQUE/À SURVEILLER/INFO calculées à la volée (lacune persistante ≥ 21 jours, baisse de moyenne mensuelle par matière, baisse de temps passé $\geq 20\%$ sur la semaine), et suggestion d'action dérivée par règle simple (pas d'appel IA dédié).
- §2.2.3 (nouveau) / §4.1 — Préférences de notification parent ajoutées (canal SMS/WhatsApp, fréquence hebdo/mensuel/critique uniquement) via une nouvelle entité NotificationPreference. N'affecte ni l'OTP ni le rappel de renouvellement (toujours envoyé).
- §1.2 / §3 / §5.5 — WhatsApp réintroduit au MVP, mais strictement comme canal de notification sortante (résumés de progression, rappels) via WhatsApp Business API — distinct du mode allégé interactif, qui reste repoussé en Phase 7 (post-MVP). L'effort de cette réintroduction est comparable à l'ajout d'un second fournisseur SMS, sans flux conversationnel entrant à gérer.
- §2.1 / §4.3 — Quiz ciblé à la demande ajouté, distinct du quiz journalier automatique : déclenché manuellement depuis "Mes lacunes" sur une notion précise. Nouveaux champs Quiz.origine (JOURNALIER/CIBLE) et Quiz.lacuneCibleId.

v1.11 → v1.12

- §4.3 — Lacune.niveauMaitrise retypé d'un enum vers un pourcentage entier (0–100), pour coller à l'affichage précis des maquettes (31%, 45%, 82%...). Calcul rendu entièrement déterministe (ratio réponses correctes/total sur la notion), sans appel IA.
- §6.1 / §6.2 — AIProvider.affinerLacunes() retirée : l'appel Sonnet qu'elle représentait n'était plus nécessaire une fois niveauMaitrise rendu déterministe — nouvelle économie de coût après celle déjà réalisée en v1.5.

v1.12 → v1.13

- §1 / §2.1 / §4.1 / §4.2 — Eleve.filiere retypée : n'était modélisée que comme un binaire Générale/TI, incohérent avec l'affichage "Série D" déjà présent dans les maquettes depuis la première revue. Devient un enum à quatre valeurs (A, C, D, TI), reflétant les séries réelles du système camerounais en 1ère/Terminale, sélectionnées à plat sans étape intermédiaire "générale puis série". Matiere.filiereRequise mis à jour en conséquence (TI reste la seule valeur utilisée pour gater Système d'Information/Programmation/Réseau ; Maths/Physique/Français/Philo restent communes aux 4 séries, sans différenciation de contenu par série pour l'instant).

v1.13 → v1.13.1

- §4.1 — Périmètre des séries confirmé : A/C/D/TI uniquement pour cette première version, décision assumée. D'autres séries camerounaises (E, F...) seront ajoutées plus tard, par simple extension de l'enum, sans changement de structure.

v1.13.1 → v1.14

- §2.1 / §2.3 / §4.3 / §4.7 — Confirmé : le contenu des épreuves diffère réellement d'une série à l'autre (A/C/D/TI), pas seulement pour les matières TI — ex. Maths Terminale C ≠ Maths Terminale D. Epreuve.filiere (déjà présent depuis la v1.0 mais jamais utilisé comme critère de filtrage actif) devient un vrai filtre : la banque d'épreuves d'un élève de 1ère/Terminale ne montre que les épreuves de sa propre série. La saisie de la série devient obligatoire côté admin pour toute épreuve de 1ère/Terminale. Nouvel index composite Epreuve(classe, filiere, matiereld).

v1.14 → v1.15

- §4.2.2 — Confirmé et explicité : contrairement à Epreuve, ExempleCorrection ne porte pas de champ filiere — le barème méthodologique officiel (structure de dissertation, répartition des points pour Français/Philo) est identique quelle que soit la série ; seul le contenu des épreuves (sujets, énoncés) varie par classe et par série. Un même ExempleCorrection sert de référence few-shot pour toutes les séries d'une matière/type d'exercice donné.

v1.15 → v1.16

- §2.9 (nouveau) / §4.1 / §3.1 / §4.7 / §7 / §10 — Mécanisme technique de rétention et suppression des données ajouté, jusque-là absent malgré une politique déjà définie dans la Politique de Confidentialité (Article 9). Cycle de vie ACTIF → INACTIF_NOTIFIE → ANONYMISE piloté par de nouveaux champs sur Eleve (statutCompte, derniereActiviteLe, dateNotificationInactivite, dateAnonymisation), deux jobs BullMQ hebdomadaires (détection d'inactivité à 6 mois, anonymisation après 60 jours de grâce sans réaction) et un job annuel d'archivage des photos de copies en fin d'année scolaire. L'anonymisation supprime le contenu pédagogique identifiable mais conserve la ligne Eleve (anonymisée) pour l'intégrité référentielle avec Abonnement/Paiement, soumis à une durée légale de conservation distincte.

v1.16 → v1.17

- §2.7 / §3 — Contradiction levée : la session élève suit désormais explicitement la même règle que les autres rôles (access token à durée de vie courte, refresh token long avec rotation) — la formulation précédente ("JWT longue durée") suggérait à tort un access token littéralement long, en contradiction avec la règle générale déjà posée au §7.
- §7 — Nouvelle ligne "Chiffrement en transit" ajoutée à la synthèse sécurité : TLS obligatoire sur tous les échanges, absent de la synthèse jusqu'ici bien qu'exigé explicitement par le référentiel de sécurité source.

v1.17 → v1.18

- §3 / §8 — Dernières mentions isolées "envoi SMS" (sans WhatsApp) corrigées pour cohérence avec la réintroduction de WhatsApp comme canal de notification (v1.11) : ligne "File d'attente/jobs asynchrones" du §3 et ligne "Synchrone/asynchrone" du §8.

- §5.3 — Ajout d'un point à vérifier dès l'obtention de l'accès CamerPay : existence d'une liste d'IPs sources autorisées pour les webhooks, à utiliser en défense en profondeur en plus de la signature HMAC (déjà couverte au §5.4) si CamerPay la propose.

v1.18 → v1.19

- §1.2 / §2.1 / §4.2 / §4.2.2 — Philosophie retirée de la banque d'épreuves et de la correction automatisée pour 1ère/Terminale, remplacée par Chimie (`banqueDisponible=true`, `classesConcernees={1ère, Terminale}`, `filiereRequise=null` — confirmé v1.20 : appliqué uniformément aux 4 séries A/C/D/TI, sans exception pour la série A). Philosophie reste accessible via le chat généraliste (§2.1), simplement sans banque d'épreuves ni correction dédiée désormais. Le type `ExempleCorrection DISSERTATION_PHILO` devient inutilisé en pratique dans ce périmètre, conservé dans l'enum sans coût de maintenance significatif.

v1.19 → v1.20

- §4.2 — Confirmé : le remplacement de Philosophie par Chimie (v1.19) s'applique bien uniformément aux 4 séries A/C/D/TI de 1ère/Terminale, y compris la série A — hypothèse levée, plus de différenciation à prévoir sur ce point. Répercuté dans les documents juridiques (CGU, Politique de Confidentialité, Mentions Légales) et signalé pour mise à jour des maquettes.

v1.20 → v1.21

- §1.2 / §2.1 / §4.2 — Exception introduite : la série D ne suit pas la règle générale Chimie pour 1ère/Terminale — elle garde SVT à la place, comme la 3ème. Chimie reste réservée aux séries A, C et TI. `Matiere.filiereRequise` retypé d'un enum simple vers un tableau (sous-ensemble de {A,C,D,TI}) pour exprimer ce cas : Chimie = {A,C,TI}, SVT = {D} — cette contrainte de série ne s'appliquant qu'aux classes où une filière existe (1ère/Terminale) ; en 3ème, sans filière, SVT reste accessible sans condition dès que la classe est dans `classesConcernees`.

v1.21 → v1.22

- §1.2 / §2.1 / §4.2 / §4.2.2 — Correction complète du mapping matières/séries pour 1ère/Terminale, remplaçant les versions v1.19-v1.21 (qui contenaient une erreur) : Série A = Maths, SVT, Français, Philosophie (Philosophie réintroduite dans la banque/correction, réservée à cette série) ; Séries C et D = Maths, Physique, Français, Chimie (identiques) ; Série TI = Maths, Physique, Français, Chimie, Système d'Information, Programmation, Réseau. 3ème inchangée = Français, Maths, Physique, SVT. Nouvelle table de référence ajoutée au §2.1. Pattern symétrique noté : Physique/Chimie partagent `filiereRequise={C,D,TI}` ; SVT/Philosophie partagent `filiereRequise={A}`. Le type `ExempleCorrection DISSERTATION_PHILO` redevient actif (série A).

v1.22 → v1.23

- §4.3 — `TentativeEpreuve.note` retiré : redondant avec `CorrectionDetail.note/noteOverride` (source de vérité unique depuis v1.1), sans mécanisme de synchronisation spécifié — risque réel d'affichage divergent entre écrans. La note est désormais systématiquement lue depuis `CorrectionDetail`.
- §2.5 / §4.6 — `UsageIA.typeUsage` enrichi de `VIDEO_FILTRAGE` : le filtrage IA des résultats YouTube (§2.5) appelle Claude Haiku, coût jusqu'ici absent du système censé surveiller tous les coûts IA (§6.4).

eleveld devient nullable sur UsageIA pour ce cas (filtrage mutualisable par notion, pas systématiquement attribuable à un seul élève).

- §2.9 / §4.6 — AuditLogSecurite enrichi de trois événements de cycle de vie du compte (COMPTE_INACTIF_DETECTE, COMPTE_ANONYMISE_AUTO, COMPTE_ANONYMISE_MANUEL) : le mécanisme de rétention (§2.9) n'était pas traçable jusqu'ici — sans ces entrées, impossible de prouver quand et pourquoi un compte a changé de statut, ni si c'était automatique ou déclenché par le parent. ip devient nullable (événements parfois issus d'un job cron, sans requête HTTP associée).

v1.23 → v1.24

- §4.2.3 (nouveau) / §6.2 / §6.4 / §7 / §2.3 / §10 — Nouvelle entité ProgrammeOfficiel : le document supposait déjà l'existence d'un "programme officiel" à plusieurs endroits (prompt caching en correction §6.4, contexteMatiere du chat §6.2, cadrage "hors programme" §7) sans jamais définir où ce contenu était stocké. Comble ce vide à partir des programmes officiels rassemblés (Maths, Chimie, Physique, Français, Philosophie par matière/classe/série). Double usage : alimente le contexte système des appels IA, et devient la taxonomie de référence des notion (Lacune, pointsManques, Video), jusqu'ici sans source de vérité formelle.

v1.24 → v1.25

- §1.2 / §2.1 / §4.2 / §4.2.2 / §10 — Correction du socle 3ème : Physique seule n'existe pas au programme camerounais de 3ème — remplacée par PCT (Physique-Chimie-Technologie), matière combinée propre à cette classe. Nouvelle entrée Matiere (classesConcernees={3ème} exclusivement, filiereRequise=null, banqueDisponible=true, correction par barème comme les autres matières scientifiques). Physique.classesConcernees perd la 3ème (désormais 1ère/Terminale uniquement). SVT reste distincte de PCT en 3ème — les deux coexistent pour cette classe.

v1.25 → v1.26

- §2.1 / §4.3 / §4.2.3 — Précision explicite ajoutée : classe est un différenciateur de contenu tout aussi réel que série, pas seulement une métadonnée de tri — confirmé par les trois matières TI (Système d'Information, Programmation, Réseau), dont le programme officiel diffère entre 1ère et Terminale. Structuellement déjà vrai depuis la v1.0 (Epreuve.classe), mais jamais qualifié explicitement comme critère de contenu, contrairement à filiere (v1.14). ProgrammeOfficiel.classe n'est jamais optionnelle, contrairement à filiere qui peut être null.

v1.26 → v1.27

- §2.3 / §4.2.3 — Périmètre exact des programmes officiels à charger au démarrage confirmé à **9 couples matière/classe/série** (et non 8, chiffre informel corrigé) : Français/Maths/PCT/SVT pour la 3ème (1 couple, la 3ème n'ayant pas de série) + Maths/Physique/Français/Chimie-ou-SVT/Philosophie pour chacune des 4 séries A/C/D/TI en 1ère et en Terminale. Les 9 fichiers JSON de programmes officiels déjà consolidés par matière/classe/série (Troisième ; Première et Terminale × A/C/D/TI) constituent la source d'import prête à charger dans `ProgrammeOfficiel.contenuStructure` (§4.2.3) — la Troisième compte comme un périmètre à part entière au même titre que chaque série de 1ère/Terminale, et non comme un cas annexe.
- §4 (schéma complet) — `schema.prisma` généré intégralement à partir de la section 4 du présent document : **26 modèles**, **22 enums**, couvrant l'ensemble des domaines décrits (§4.1 comptes/authentification, §4.2 contenu pédagogique, §4.2.2 exemples de correction, §4.2.3 programmes officiels,

§4.3 suivi pédagogique, §4.4 chat-tuteur, §4.5 abonnement/paiement, §4.6 monitoring/sécurité), avec tous les index prioritaires du §4.7 implémentés. Écarts notables par rapport à un premier brouillon antérieur au présent document : suppression de tout modèle relationnel Module/Thème au profit de `ProgrammeOfficiel.contenuStructure` (Json), conformément à la décision déjà actée en §4.2.3 ; suppression des modèles `Classe / Serie` au profit des enums `NiveauClasse / Filiere` portés directement par `Eleve`, `Epreuve` et `ProgrammeOfficiel` ; relation Parent ↔ Élève modélisée en N—N via `ParentEleveLink` (cohérent avec l'inscription autonome de l'élève, §1.2/§2.2), avec ajout d'une contrainte d'unicité (`parentId, eleveId`) non explicitement listée en §4.7 mais nécessaire à l'intégrité du lien vérifié (§4.1).

- **§4.7** — Contrainte technique documentée : l'unicité composite `ProgrammeOfficiel(matiereId, classe, filiere)` doit être surveillée en seed, Postgres traitant `NULL` comme distinct dans une contrainte unique (plusieurs lignes `filiere = NULL` pour un même couple matière/classe resteraient techniquement possibles sans garde applicative supplémentaire).
- **§10** — Validation du schéma effectuée manuellement (équilibre syntaxique, cohérence de chaque relation déclarée des deux côtés) faute d'accès réseau au moteur Prisma dans l'environnement de rédaction de ce document ; validation formelle (`prisma validate`, `prisma migrate dev`) à effectuer en premier dans Claude Code, avant toute autre étape de la Phase 0.

v1.27 → v1.28

- §2.1 (nouveau §2.1.1) — Exigences UI de différenciation Tuteur IA / Correction IA ajoutées : la distinction fonctionnelle entre chat général (mode 1), chat contextualisé à une épreuve (mode 2, §4.4) et correction IA (§6.2) n'était assortie d'aucune exigence de nomenclature, d'emplacement dans la navigation ou d'icône — écart relevé lors d'une revue des maquettes (docs/maquettes/screenshots/), le mode 2 n'y disposant d'aucun point d'entrée visuel. Nouveau §2.1.1 : bandeau contextuel obligatoire en mode 2, bouton dédié sur l'écran de résultat de correction, icône Tuteur IA reprise sur la landing page, icône de correction jamais réutilisée pour une surface de chat.

v1.28 → v1.29

- §2.1 / §4.2 (§4.2.3) — SVT ajoutée à la banque d'épreuves et à la correction automatisée pour les séries C, D et TI (1ère et Terminale), en complément des matières déjà prévues et non en remplacement. Le Français n'est pas retiré : l'ajout centralise la préparation sur les matières scientifiques pour C/D/TI, tout en conservant le Français, obligatoire à l'examen national (BEPC / Probatoire / Baccalauréat). Table de référence §2.1 mise à jour. Le socle passe de 9 à 15 couples matière/classe/série (§4.2.3) : les 6 sections svt des programmes officiels C, D et TI (1ère + Terminale), déjà consolidées, sont désormais chargées dans `contenuStructure`. `Matiere.filiereRequise` de SVT passe de {A} à {A, C, D, TI}, dérivé automatiquement par le seed. Aucun changement de schéma Prisma.

v1.29 → v1.30

- §4.2.2 / §4.3 — Nouveau type d'exercice méthodologique `COMMENTAIRE_COMPOSE` ajouté à l'enum `TypeExerciceCorrection` (migration Prisma `add_commentaire_compose_type_exercice`). Cinquième barème officiel Français/Philo, à côté de `DISSERTATION_PHILO`, `DISSERTATION_LITTERAIRE`, `CONTRACTION_TEXTE` et `DISCUSSION`. Répartition : Introduction 3 · Développement 12 (2 axes de lecture, méthode O.C.I.E. — Observation, Citation, Interprétation, Effet — environ 5,5 pts par axe + transitions 1) · Conclusion 3 · Présentation & langue 2 = 20 pts ; épreuve de Français. Le pipeline `corrigerCopie()` l'injecte en few-shot comme les autres via `ExempleCorrection` (résolu par `matiereId + typeExercice`). Aucun autre changement de schéma ; §4.2.2 mis à jour (5 barèmes officiels).

v1.30 → v1.31

- §4.2.2 / §4.3 — Deux types d'exercice propres à la classe de 3ème (Français) ajoutés à l'enum `TypeExerciceCorrection` (migration Prisma `add_expression_ecrite_correction_orthographique_type_exercice`) : `EXPRESSION_ECRITE` et `CORRECTION_ORTHOGRAPHIQUE`. Ils ne concernent ni la 1ère ni la Terminale. `EXPRESSION_ECRITE` suit une grille pondérée par critères sur 10 points (Pertinence de la production 3 · Cohérence 3 · Correction de la langue 3 · Présentation 1), doublée lorsque l'épreuve est notée sur 20 — même logique que les autres barèmes méthodologiques. `CORRECTION_ORTHOGRAPHIQUE` suit au contraire un mécanisme de comptage de fautes, fondamentalement distinct des barèmes pondérés par critères : l'épreuve dissimule une vingtaine de fautes, 1 point par faute identifiée et corrigée, le point n'étant acquis que si deux conditions indissociables sont réunies pour le mot concerné (rayer proprement le mot erroné et écrire l'orthographe exacte au-dessus) ; un mot déjà correct rayé à tort est pénalisé. Comme pour `COMMENTAIRE_COMPOSE` en v1.30, la table visuelle §4.2.2 n'est pas étendue (elle provoquerait une repagination) : le contenu normatif est l'enum `TypeExerciceCorrection` et la présente entrée. §4.2.2 compte désormais 7 barèmes officiels ; le pipeline `corrigerCopie()` les injecte en few-shot via `ExempleCorrection` (résolu par `matiereld + typeExercice`), inchangé.

v1.31 → v1.32

- §1, §1.2, §2.4, §2.6, §3, §4.5, §5, §6, §8, §9, §10 — CamerPay remplacé par NotchPay comme fournisseur Mobile Money : CamerPay n'a jamais eu d'accès réel obtenu malgré son statut de fournisseur nommé depuis les premières versions de ce document ; NotchPay offre un accès sandbox immédiat avec couverture MTN et Orange Mobile Money au Cameroun. PAYMENT_MODE passe de mock|sandbox|live à mock|notchpay : NotchPay n'expose qu'une seule URL d'API (api.notchpay.co) pour le test et le réel, distingués uniquement par le préfixe de la clé publique (pk_test_... / pk_live_...) — il n'existe donc plus de classe séparée CamerPaySandboxProvider/CamerPayLiveProvider, une seule NotchPayProvider couvre les deux. Flux d'initiation réel en 2 appels : POST /payments crée la transaction (montant, devise, téléphone) et renvoie un id de transaction et une authorization_url de repli ; POST /payments/{transaction} avec un canal (cm.orange/cm.mtn) déclenche l'invite USSD/app sur le téléphone du payeur — jamais de confirmation synchrone, toujours par webhook. Vérification webhook : header x-notch-signature, HMAC-SHA256 du corps JSON brut avec le 'Hash Key' du dashboard NotchPay (distinct des clés API publique/secrète). Nouveau champ Payeur.opérateur (ORANGE/MTN) sur l'interface PaymentProvider — jusqu'ici l'opérateur Mobile Money choisi par l'utilisateur était capturé côté route mais jamais transmis au provider, bloquant pour sélectionner le canal NotchPay ; ignoré par MockPaymentProvider. Paiement.referenceCamerPay renommé Paiement.referenceTransaction (migration Prisma dédiée, colonne renommée sans perte de données) ; WebhookLog.provider n'a plus de défaut 'CAMERPAY' — posé explicitement (MOCK/NOTCHPAY) à chaque écriture. Nouvelle fonction paiementsSontReels() : distingue l'argent réel de l'argent de test via le préfixe de clé (PAYMENT_MODE seul ne suffit plus, notchpay pouvant tourner en sandbox comme en argent réel) ; pilote désormais le bandeau 'données de test' des écrans admin Paiements/Revenus. Limite connue : NotchPayProvider est implémenté contre la documentation publique NotchPay, non encore exercé contre un vrai compte sandbox au moment de cette entrée — l'hypothèse que l'id de transaction retourné à l'initiation correspond au data.id reçu dans le webhook n'est pas confirmée noir sur blanc par leur documentation et reste à valider au premier paiement réel. Comme pour COMMENTAIRE_COMPOSE en v1.30, le bandeau paiement de la page de garde et la table §3 (stack technique) sont mis à jour par remplacement de mot, sans reflow ; le détail mécanique complet (flux HTTP, en-têtes, limite sandbox) réside dans cette entrée plutôt que dans le corps de la section 5, qui reste volontairement concis.

1. Présentation du projet

Klarity est une plateforme éducative destinée aux élèves camerounais de 3ème, 1ère et Terminale (séries A, C, D et TI pour 1ère/Terminale), accessible en version web (site responsive), pensée pour fonctionner en environnement à connectivité limitée (design mobile-first, poids de page réduit) et avec un moyen de paiement local (Mobile Money, via NotchPay). Un mode allégé WhatsApp est envisagé pour une phase ultérieure (cf. §1.2, §10) mais n'entre pas dans le périmètre du MVP.

Le projet répond à trois besoins distincts, portés par trois interfaces séparées : un espace élève (chat-tuteur IA, banque d'épreuves, correction automatisée, quiz journalier), un espace parent (suivi de la progression, des lacunes et du temps passé) et un espace admin (gestion de contenu, supervision des utilisateurs et du chiffre d'affaires).

Objectif de ce document. Fournir une base de référence unique — fonctionnelle et technique — combinant le brief produit initial, le plan de scalabilité et le référentiel de sécurité déjà validés, enrichie du choix de stack, du modèle de données complet (Docker-ready) et de l'architecture d'intégration pour NotchPay et l'API Claude, dans un contexte où aucun des deux accès (paiement live, clé API Claude) n'est encore disponible.

1.1 — Acteurs et interfaces

Acteur	Interface	Fonctions principales
Élève	Web (responsive) · WhatsApp allégé — post-MVP, cf. §1.2	Chat-tuteur IA contextualisé à l'épreuve en cours (cf. §2.1), banque d'épreuves (PDF), upload photo de copie pour correction IA, notes et corrigés, quiz journalier ciblé sur les lacunes, vidéos YouTube recommandées.
Parent	Web (desktop/mobile)	Connexion par code élève + téléphone + OTP SMS, suivi des notes et

Acteur	Interface	Fonctions principales
		lacunes par matière, temps passé par jour sur la plateforme, historique de progression.
Admin (fondateur)	Back-office web	Statistiques d'usage (élèves/parents, actifs en temps réel), chiffre d'affaires (jour/mois/année/personnalisé), ajout d'épreuves par classe/filière/matière. Le catalogue vidéo est entièrement automatisé (cf. §2.5) — aucune intervention admin, pas même de supervision.

1.2 — Contraintes structurantes

- Une dépendance externe reste à valider en conditions réelles : NotchPay (§5), code déjà écrit. Une autre n'est pas encore disponible : la clé API Anthropic Claude — le document prévoit une architecture d'abstraction permettant de développer et tester sans bloquer sur ces accès (section 5 et 6).
- Public mineur : la quasi-totalité des données traitées concerne des enfants/adolescents — contrainte transversale sur la sécurité et la conformité (Loi n°2000/011/2024/017).
- Connectivité limitée : design mobile-first, poids de page réduit sur la version web — couvre l'essentiel de la contrainte au MVP. Le mode allégé WhatsApp interactif (l'élève utilisant WhatsApp comme interface complète — upload photo, quiz, chat contextualisé à l'épreuve, affichage de correction), initialement envisagé en parallèle, est repoussé en post-MVP (cf. §10) : la réplication de chaque flux produit en interface conversationnelle représente un effort de développement distinct et non négligeable, à ne pas mener en parallèle du cœur du MVP. Distinction v1.11 : WhatsApp comme simple canal de notification à sens unique (résumés de progression, rappels de renouvellement, cf. §2.2.2, §5.5) reste au MVP — l'effort est comparable à l'intégration d'un second fournisseur SMS (messages-modèles sortants via WhatsApp Business API), sans flux conversationnel à construire ; ce n'est pas la complexité qui a motivé le report du mode allégé.
- Modèle économique sensible aux coûts IA : le plan premium à usage "illimité" doit rester rentable — nécessite un contrôle strict des coûts par utilisateur dès la conception (section 6.4).
- Extension anglophone prévue après validation : le modèle de données inclut un champ de langue/ localisation dès le MVP pour éviter une refonte de schéma plus tard.
- **NOUVEAU v1.1, réécrit v1.22.** Périmètre matières du MVP volontairement restreint pour la banque d'épreuves et la correction automatisée, avec un socle qui varie par classe et par série, cf. table de référence au §2.1. Anglais et Histoire-Géographie restent hors scope pour la banque d'épreuves et la correction — exclusion assumée, pas un oubli de spécification. Le chat-tuteur suit une règle différente (cf. §2.1) : il est généraliste et couvre l'ensemble du programme scolaire de la classe/filière de l'élève, y compris les matières sans banque d'épreuves.
- **NOUVEAU v1.2.** Inscription élève autonome, sans validation parentale préalable : l'élève crée son compte seul (§2.1), le lien parent ↔ élève se fait a posteriori via ParentEleveLink (§2.2) — choix de produit assumé pour réduire la friction, pas un oubli de conformité. Concrètement, cela signifie qu'un élève peut créer son compte, effectuer des tentatives d'épreuves, utiliser le chat-tuteur et consulter ses corrections sans qu'aucun parent ne soit informé ni n'ait donné son accord, tant que le lien parent n'est pas établi. Ce choix est documenté ici explicitement comme une décision de produit assumée, et non comme un oubli de conformité.

- **NOUVEAU v1.2.** Devise et paiement : Klarity reste 100% XAF/NotchPay, aucune gestion diaspora ni multi-devise n'est prévue — cible exclusivement les familles au Cameroun (à la différence d'autres projets du même porteur gérant une audience diaspora).

2. Périmètre fonctionnel détaillé

2.1 — Parcours élève

Module	Description fonctionnelle
Inscription	Nom, classe (3ème/1ère/Tle), filière — série A, C, D ou TI, pour 1ère/Tle uniquement (la 3ème n'a pas de filière) → génération d'un code unique format ELE-XXX-XXX, communiqué à l'élève pour transmission au parent. Définition d'un PIN à 4 chiffres à la première connexion (cf. §2.7). L'élève crée son compte seul, sans validation parentale préalable — le lien avec le parent se fait a posteriori, dès que celui-ci se connecte avec le code transmis (cf. §2.2).
Chat-tuteur IA	Généraliste — couvre l'ensemble du programme scolaire de la classe/filière de l'élève, y compris les matières sans banque d'épreuves ni correction IA (Anglais, Histoire-Géo), et pas seulement les matières listées dans la table de référence ci-dessous. Deux modes coexistent, tous deux strictement cadrés au programme scolaire de l'élève (aucune dérive hors sujet scolaire, cf. §7) : (1) chat par matière — l'élève choisit une matière (y compris hors banque d'épreuves) et pose une question générale sur une notion de cours, sans épreuve précise en cours de traitement ; (2) chat contextualisé à une épreuve — uniquement disponible pour les matières avec banque d'épreuves, lorsque l'élève consulte une épreuve ou une correction, l'énoncé et le corrigé sont injectés comme contexte du prompt. Dans les deux cas, réponse détaillée générée par IA, accompagnée de vidéos YouTube adaptées à la notion, intégrées via iframe (cf. §3 et §4.2) — le pipeline vidéo (§2.5) fonctionne par notion, indépendamment de la présence d'une banque d'épreuves pour la matière. Toute question hors du programme scolaire de l'élève fait l'objet d'un refus poli et d'une redirection (cf. §7). Modèle : Claude Haiku (section 6).
Banque d'épreuves	Sélection d'une épreuve par matière. Le socle dépend de la classe et, pour 1ère/Terminale, de la série (voir table de référence ci-dessous). Anglais et Histoire-Géo ne sont couvertes par aucune série au MVP — accessibles uniquement via le chat généraliste (§2.1, "Chat-tuteur IA"). Pour 1ère/Terminale, le filtrage se fait aussi par série sur les épreuves elles-mêmes (§4.3) : un élève ne voit que les épreuves correspondant à sa propre série — les épreuves diffèrent réellement de contenu d'une série à l'autre, pas seulement pour les matières TI (ex. Maths Terminale C ≠

Module	Description fonctionnelle
	Maths Terminale D). De la même façon (précision v1.26), le contenu diffère réellement d'une classe à l'autre pour une même matière/série — 1ère et Terminale ne sont pas deux occurrences du même programme : les trois matières TI (Système d'Information, Programmation, Réseau) ont chacune un programme officiel distinct en 1ère et en Terminale, exactement comme Epreuve.classe le permettait déjà structurellement depuis la v1.0, mais sans que ce soit dit explicitement jusqu'ici. Téléchargement PDF, traitement papier, upload photo.
Correction automatisée	L'IA analyse la copie photographiée — upload multi-pages : l'élève photographie chaque page de sa copie séparément (cf. §4.3), toutes les pages sont analysées ensemble par l'IA comme une seule copie continue —, identifie ce qui n'a pas été trouvé, calcule une note, puis fournit le corrigé complet. Nombre de tentatives illimité par épreuve — la correction produite lors de la première tentative fait foi : les tentatives suivantes (retraitement) réaffichent cette même correction, sans nouvel appel au modèle IA (cf. §4.3 et §6.4 pour la justification coût). Chaque point manqué identifié par la correction crée ou met à jour immédiatement une Lacune pour l'élève (cf. "Détection de lacunes" ci-dessous et §4.3), ce qui déclenche à son tour le pipeline vidéo automatisé (§2.5) : les vidéos de renforcement correspondantes apparaissent directement sur l'écran de correction, dès qu'elles sont prêtes. Si l'élève estime cette correction manifestement erronée, il peut la signaler pour revue admin (cf. §2.8). Modèle : Claude Sonnet (section 6).
Détection de lacunes	Immédiate, à chaque correction : chaque point manqué identifié dans une correction (CorrectionDetail.pointsManques) crée ou met à jour directement une Lacune pour l'élève sur la notion concernée — pas d'attente d'un pattern répété. niveauMaitrise est un pourcentage calculé de façon déterministe (ratio réponses correctes/total observées sur cette notion, cf. §4.3), sans appel IA, recalculé à chaque nouvelle donnée (correction ou quiz) ; resolu passe à true quand ce pourcentage dépasse un seuil sur un nombre minimum de réponses récentes.
Quiz journalier	Génération automatique d'un quiz quotidien ciblé sur les lacunes actives, avec vidéos de renforcement associées à chaque notion faible. Distinct du quiz ciblé à la demande (bouton "Commencer le quiz associé" depuis l'écran "Mes lacunes") : l'élève peut déclencher manuellement un quiz portant uniquement sur une lacune précise, à tout moment, en plus du cycle automatique quotidien (cf. §4.3).
Compte à rebours examen	Widget affiché sur le dashboard élève (et parent) indiquant le nombre de jours restants avant l'examen national correspondant à la classe de l'élève : BEPC (3ème), Probatoire (1ère), Baccalauréat (Terminale). Affiche un

Module	Description fonctionnelle
	compte en jours si la date officielle est connue, sinon une période estimée (cf. §2.3 et §4.2).

Table de référence — socle matières banque d'épreuves + correction (v1.29) :

Classe / Série	Matières banque + correction
3ème (pas de filière)	Français, Maths, PCT (Physique-Chimie-Technologie), SVT
1ère/Terminale — Série A	Maths, SVT, Français, Philosophie
1ère/Terminale — Série C	Maths, Physique, Français, Chimie, SVT
1ère/Terminale — Série D	Maths, Physique, Français, Chimie, SVT
1ère/Terminale — Série TI	Maths, Physique, Français, Chimie, Système d'Information, Programmation, Réseau, SVT

Cette table couvre désormais **15 couples matière/classe/série** au sens de ProgrammeOfficiel (§4.2.3, rév. v1.29) : le socle de 9 (3ème + 4 séries × 2 classes) plus 6 couples SVT pour les séries C, D et TI (1ère et Terminale). Le Français reste au socle — SVT est un complément, pas un remplacement.

2.1.1 — Exigences UI de différenciation Tuteur IA / Correction IA (nouveau v1.28)

- Nomenclature — quand contexteEpreuve est fourni à chat() (mode 2, §4.4), l'écran de chat doit afficher un bandeau contextuel explicite ("Discussion à propos de : [nom de l'épreuve]"), visuellement distinct de l'en-tête générique "Tuteur IA · En ligne" utilisé en mode 1. Aucune maquette actuelle ne montre ce bandeau — à ajouter à la prochaine revue de maquettes.
- Emplacement dans la navigation — Tuteur IA et Épreuves/Correction restent deux items de navigation top-level séparés (déjà respecté dans les maquettes actuelles). L'écran de résultat de correction (docs/maquettes/screenshots/08_resultat_correction_detaillée.png) doit exposer un troisième bouton d'action, "Discuter de cette copie", à côté de "Recommencer l'épreuve" et "Signaler cette correction", ouvrant le Tuteur IA en mode 2 avec contexteEpreuve pré-rempli sur l'épreuve consultée.
- Icônes — l'icône Tuteur IA (pastille verte, glyphe étoile) utilisée dans la navigation applicative et sur l'écran de chat doit être reprise à l'identique sur le widget de démonstration de la landing page (actuellement affiché sans icône, seul le libellé texte "Tuteur IA" figure). L'icône robot utilisée à l'étape 2 du parcours "Comment ça marche" (correction IA) reste réservée exclusivement à la Correction IA — jamais réutilisée sur une surface de chat, pour ne pas laisser croire que le Tuteur IA produit lui-même une correction chiffrée.

2.2 — Parcours parent

- Connexion par code élève (ELE-XXX-XXX) + numéro de téléphone → réception d'un code de vérification (OTP) par SMS.
- Accès à un dashboard lecture seule : évolution par matière, lacunes actives/résolues, temps passé sur la plateforme par jour.
- Aucune action de gestion de contenu ou de compte élève côté parent dans le périmètre MVP, à l'exception du paiement de l'abonnement (cf. §2.6) et de la réinitialisation du PIN élève (cf. §2.7).
- Export PDF du rapport mensuel — bouton à la demande sur le dashboard parent, générant un PDF reprenant les indicateurs déjà affichés (progression générale, notes des épreuves corrigées, lacunes actives/résolues, temps passé) sur le mois en cours ou le dernier mois complet. Généré à la volée à partir des données existantes (pas de nouvelle donnée à collecter), sans job asynchrone ni stockage persistant du PDF lui-même — régénérable à l'identique à chaque demande.
- Lien a posteriori — l'élève ayant déjà créé son compte de manière autonome (cf. §2.1), la connexion parent ne fait qu'établir le ParentEleveLink déjà prévu au schéma (§4.1) ; ce n'est pas une étape de validation préalable à l'usage de la plateforme par l'élève.

2.2.1 — Sélection de l'enfant suivi (parent multi-enfants)

Le schéma prévoit déjà qu'un parent puisse suivre plusieurs enfants (relation N—N via ParentEleveLink, cf. §4.1), mais l'interface doit exposer explicitement ce cas.

- Si le parent n'a qu'un seul ParentEleveLink actif, le dashboard s'ouvre directement sur cet enfant, sans sélecteur.
- S'il a plusieurs liens actifs, un sélecteur d'enfant est affiché en haut du dashboard, permettant de basculer d'un enfant à l'autre sans se reconnecter.
- Le choix effectué est mémorisé dans Parent.dernierEleveConsulted, mis à jour à chaque changement de sélection.

- À la connexion suivante, le dashboard s'ouvre directement sur ce dernier enfant consulté, plutôt que de forcer un nouveau choix à chaque session.

NOUVEAU v1.11

2.2.2 — Alertes intelligentes et suggestions d'action

Le dashboard parent affiche des alertes classées par niveau de gravité, calculées à la volée au chargement du dashboard à partir des données déjà existantes (Lacune, TentativeEpreuve, SessionActivite) — aucune donnée ni job asynchrone supplémentaire nécessaire pour le MVP, ces règles étant des agrégations simples sur des volumes par famille (quelques dizaines d'enregistrements).

- CRITIQUE — une Lacune reste active (resolu = false) depuis plus de 21 jours (3 semaines) sans amélioration de niveauMaitrise depuis sa détection.
- À SURVEILLER — la moyenne des notes d'une matière sur le mois en cours est en baisse par rapport au mois précédent ; ou le temps passé cumulé (SessionActivite) sur la semaine en cours est en baisse d'au moins 20% par rapport à la semaine précédente.
- INFO — progression stable ou en hausse sur une matière sur le mois ; nombre de quiz complétés sur la semaine.

Les seuils (21 jours, 20%) sont des valeurs de départ raisonnables, non figées — à ajuster après observation des premiers usages réels plutôt que sur-spécifiées avant lancement.

Suggestion d'action — dérivée par règle simple (pas d'appel IA dédié, pour maîtriser le coût) : l'alerte la plus critique en cours (typiquement la Lacune active la plus ancienne) alimente un bloc "Suggestion d'action" avec un texte-modèle ("Encouragez [Élève] à refaire les exercices ciblés sur [notion] cette semaine") et un lien vers le détail de cette lacune (§2.1 "Mes lacunes"). Pas de génération de texte libre par IA pour cette suggestion au MVP — à réévaluer si le rendu template s'avère trop générique après usage réel.

NOUVEAU v1.11

2.2.3 — Préférences de notification (parent)

Le parent choisit, depuis un écran de paramètres, comment être informé de la progression de son enfant — distinct du rappel de renouvellement d'abonnement (§5.5), qui reste systématique et non désactivable.

- Canal — SMS ou WhatsApp (message-modèle sortant via WhatsApp Business API, cf. §1.2 — ne nécessite pas le mode allégé interactif post-MVP). SMS reste le choix par défaut.
- Fréquence — résumé hebdomadaire, résumé mensuel, ou alertes critiques uniquement (cf. §2.2.2). Un job BullMQ planifié (hebdomadaire ou mensuel selon la préférence) compose et envoie le résumé ; les alertes critiques peuvent en plus déclencher un envoi immédiat si "alertes critiques uniquement" est sélectionné.
- Ces préférences ne s'appliquent qu'aux résumés de progression — elles n'affectent ni l'OTP de connexion (§2.2) ni le rappel de renouvellement (§5.5, toujours envoyé).

2.3 — Back-office admin

- Tableau de bord : nombre total de parents / élèves, utilisateurs actifs en temps réel.
- Chiffre d'affaires : vues jour / mois / année / période personnalisée.
- Gestion de la banque d'épreuves : ajout d'une épreuve rattachée à une classe, une matière et — pour 1ère/Terminale — une série (A/C/D/TI, cf. §4.3) → alimente directement la banque correspondante côté élève. La saisie de la série devient obligatoire pour toute épreuve de 1ère/Terminale, les contenus différant réellement d'une série à l'autre.

- Gestion des dates d'examen (BEPC / Probatoire / BAC) par année scolaire — champ date précise si connue, sinon période estimée en attendant l'annonce officielle du MINESEC. Alimente le compte à rebours affiché côté élève et parent (cf. §2.1 et §4.2).
- Corrections signalées — liste filtrée des corrections mises en revue par les élèves (cf. §2.8), avec accès à la copie, au corrigé et au détail IA, et possibilité d'override manuel de la note.
- Programmes officiels (v1.24) — chargement initial, par l'admin, du contenu structuré par module/notion pour chaque matière/classe/série du périmètre MVP (ProgrammeOfficiel, cf. §4.2.3), à partir des documents de programme officiels déjà rassemblés. **Périmètre exact confirmé v1.27 : 9 couples matière/classe/série** (Troisième + 4 séries × 2 classes en 1ère/Terminale), correspondant à 9 fichiers JSON de programmes déjà consolidés, prêts à l'import. Donnée de configuration versionnée au même titre que les barèmes Français/Philo (§4.2.2), pas une fonctionnalité à fort trafic — pas d'interface de gestion élaborée nécessaire, un simple formulaire d'import suffit au MVP.

Décision — pas d'interface admin pour le catalogue vidéo. Contrairement à une première version de ce document, le catalogue vidéo n'est pas géré manuellement par l'admin. Le pipeline est entièrement automatisé (cf. §2.5) : aucune interface de gestion, d'ajout ou de mapping vidéo ne doit être construite côté back-office.

2.4 — Modèle économique

Plan	Prix	Contenu
Gratuit	0 FCFA	2 épreuves par matière, pas de vidéos explicatives, quiz/exercices journaliers limités à 2. La correction automatisée par IA et la détection de lacunes fonctionnent normalement sur ces 2 épreuves — ce ne sont pas des fonctionnalités réservées au Premium (cf. précision ci-dessous).
Premium	5 000 FCFA/mois (3 000 FCFA en période Noël déc-jan-fév et Pâques avr-mai-juin)	Épreuves illimitées, vidéos illimitées, quiz et exercices journaliers illimités (avec plafond technique raisonnable — voir §6.4).

Paiement par Mobile Money exclusivement via l'agrégateur NotchPay, avec page dédiée de saisie du numéro Mobile Money et pop-up de confirmation lors de la vérification (voir section 5 pour l'architecture d'intégration en l'absence d'accès live, et §2.6 pour le parcours payeur). Le paiement par carte bancaire, envisagé dans une version antérieure, est retiré du périmètre : aucune saisie de données de carte (numéro, expiration, CVV) n'est prévue sur la Plateforme, ce qui évite tout risque de conformité PCI-DSS côté Klarity — seul le flux Mobile Money, déjà entièrement délégué à NotchPay, est conservé.

PRÉCISION v1.9. Ce qui distingue réellement Gratuit et Premium — la différence porte sur le volume (nombre d'épreuves, de quiz/exercices par jour) et sur les vidéos de renforcement (indisponibles en Gratuit), pas sur la qualité ou la disponibilité du cœur pédagogique. Concrètement, sur ses 2 épreuves par matière, un élève au plan Gratuit bénéficie exactement du même traitement qu'un élève Premium : correction automatisée par IA (§2.1), détection immédiate de lacunes (§2.1, §4.3) et création des Lacune correspondantes. Ce qui lui manque, c'est le pipeline vidéo de renforcement associé à ces lacunes (§2.5) — réservé au Premium — et la possibilité de dépasser 2 épreuves par matière ou 2 quiz/exercices par jour.

2.4.1 — Détermination automatique de la période tarifaire

Le prix proposé au moment de l'abonnement (5 000 ou 3 000 FCFA) est déterminé côté serveur, jamais côté client, à partir de la date du jour :

```
determinerPeriodeTarifaire(date): 'NOEL' | 'PAQUES' | 'NORMALE'  
-> NOEL si mois dans {decembre, janvier, fevrier} (3000 FCFA)  
-> PAQUES si mois dans {avril, mai, juin} (3000 FCFA)  
-> NORMALE sinon (5000 FCFA)
```

Cette fonction est appelée à deux moments distincts, avec deux conséquences différentes :

- À l'affichage de la page de paiement : détermine le prix affiché au visiteur (3 000 ou 5 000 FCFA).
- Au moment du paiement effectif : le prix retenu est figé dans Abonnement.prixApplique (cf. §4.5) et ne change plus, même si la période tarifaire change en cours d'abonnement — cohérent avec la règle déjà posée de ne jamais recalculer a posteriori depuis un tarif courant.

2.5 — Alimentation automatique du catalogue vidéo

Décision. Le catalogue vidéo n'est jamais géré manuellement. Le pipeline est entièrement automatisé (recherche YouTube + filtrage IA + cache) — zéro intervention admin, pas même de supervision. Cette décision remplace toute mention antérieure d'un back-office vidéo dans ce document.

- Déclenchement : dès qu'une nouvelle Lacune.notion apparaît sans vidéo associée en cache (LacuneVideoCache vide pour cette notion), un job asynchrone est mis en file (cf. plan de scalabilité §1, qui identifiait déjà la recherche/filtrage vidéo comme traitement à ne jamais bloquer une requête utilisateur). Depuis qu'une Lacune est créée immédiatement à chaque correction (cf. §2.1, §4.3), ce déclencheur s'active dès la fin d'une correction — les vidéos apparaissent sur l'écran de correction dès qu'elles sont prêtes, sans attendre le cycle du quiz journalier. Le même mécanisme continue par ailleurs d'alimenter le quiz journalier pour les Lacune actives plus anciennes.
- Recherche : appel à l'API YouTube Data v3 avec une requête construite à partir de la notion, de la matière et de la classe/filière (et de la langue, pour l'extension anglophone à venir).
- Filtrage IA : les résultats bruts sont passés à Claude Haiku (cf. §6.1) qui élimine les vidéos hors-sujet, non pédagogiques ou de qualité douteuse, et sélectionne les meilleures correspondances. Coût suivi via UsageIA.VIDEO_FILTRAGE (v1.23), comme les autres appels IA du système.
- Mise en cache : les vidéos retenues sont écrites dans LacuneVideoCache et exposées via l'entité Vidéo (cf. §4.2) — les appels suivants pour la même notion servent directement depuis le cache, sans re-consommer l'API YouTube ni l'appel LLM.
- Aucun CRUD admin : aucune route, aucun écran back-office n'est prévu pour créer, modifier ou valider une entrée vidéo — le pipeline est la seule source de vérité.

NOUVEAU v1.1

2.6 — Parcours de paiement : parent et élève

- Le parent est prioritaire sur le paiement de l'abonnement de son enfant, mais un élève qui a les moyens de se prendre en charge peut également initier et régler son propre abonnement — cas fréquent sur le terrain (argent de poche, Mobile Money personnel).

- La page de paiement (§2.4) est accessible depuis les deux espaces : espace parent (dashboard) et espace élève. C'est une exception explicite à la règle générale du §2.2 excluant toute action de gestion côté parent.
- Paiement.payeurTelephone capture le numéro Mobile Money utilisé, indépendamment de qui paie ; un champ payeurRole (PARENT/ELEVE) est ajouté pour distinguer qui a déclenché le paiement (cf. §4.5), utile pour les statistiques admin et l'analyse d'usage.
- Le rappel de renouvellement (§5.5) est envoyé aux deux canaux disponibles (parent si lien vérifié existe, et élève) plutôt qu'à un seul destinataire, pour ne jamais dépendre d'un seul point d'attention.
- Paiement solo par un mineur — position assumée. Un élève peut payer lui-même son abonnement premium en Mobile Money sans qu'aucun parent ne soit impliqué à aucune étape du parcours — conséquence directe de l'inscription autonome (§1.2, §2.1) combinée à l'accès paiement ouvert côté élève ci-dessus. Ce n'est pas jugé problématique dans le contexte camerounais : le Mobile Money local n'impose pas la même barrière d'âge qu'une carte bancaire, et la responsabilité de la ligne Mobile Money utilisée reste celle de son titulaire, pas de Klarity. Ce point est documenté ici explicitement comme un choix de produit assumé, prêt à être justifié auprès d'un partenaire ou d'un régulateur — pas un angle mort découvert sous pression.

NOUVEAU v1.1

2.7 — Authentification de l'élève

L'élève se connecte avec son codeEleve (ELE-XXX-XXX) associé à un PIN à 4 chiffres, mécanisme volontairement plus léger qu'un mot de passe classique (public adolescent, connectivité limitée).

- Première connexion — l'élève entre son codeEleve et définit son PIN.
- Connexions suivantes — codeEleve + PIN sur le parcours web.
- Session — même principe que pour les autres rôles (cf. §7) : access token à durée de vie courte, refresh token à durée de vie longue avec rotation, jamais l'inverse. C'est le refresh token, pas l'access token, qui reste valide sur la durée — l'élève n'a donc pas à ressaisir son PIN à chaque session sur un même appareil, sans que ça affaiblisse la fenêtre d'exposition d'un access token compromis. Précision v1.17 : la formulation "session longue durée" utilisée jusqu'ici prêtait à confusion en suggérant un access token littéralement long — corrigé pour lever toute ambiguïté avec la règle générale du §7.
- PIN oublié — réinitialisation déclenchée exclusivement par le parent depuis son dashboard (canal déjà vérifié par OTP téléphone, §2.2), pas de flux de reset autonome côté élève.
- Sécurité — rate limiting sur les tentatives de PIN (même logique que l'OTP, §7), verrouillage temporaire après plusieurs échecs, journalisation dans AuditLogSecurite (§4.6).

NOUVEAU v1.2

2.8 — Signalement d'une correction contestée

Une correction (§2.1, §4.3) fait foi dès la première tentative et n'est jamais recalculée automatiquement. Il faut donc un mécanisme pour les cas où l'IA se trompe manifestement (mauvaise lecture OCR d'une copie manuscrite, mauvaise application du barème) — sans quoi l'élève n'a aucun recours face à une note qu'il juge injuste.

- Côté élève — un bouton "Signaler cette correction" est disponible sur l'écran de correction. L'élève sélectionne un motif court (ex. lecture illisible, mauvais barème appliqué, autre) et peut ajouter un commentaire libre.

- Traitement — le signalement place la correction dans une file de revue accessible côté admin (§2.3), sans bloquer l'accès de l'élève à sa note ou à son corrigé en attendant le traitement.
- Côté admin — l'admin consulte la copie photographiée, le corrigé de référence et le détail de la correction IA (points forts/manqués), puis peut soit rejeter le signalement (correction confirmée), soit forcer manuellement une nouvelle note avec une justification associée. Aucun nouvel appel au modèle IA n'est déclenché par cette revue — c'est un override humain, pas un retraitement automatique.
- Une seule résolution par signalement — une fois traité (rejeté ou override), le signalement est clos ; l'élève peut ressignaler si un nouveau problème apparaît, mais pas rouvrir indéfiniment le même signalement.
- Volumétrie attendue faible — mécanisme pensé comme un filet de sécurité, pas comme un canal de contestation systématique ; aucun SLA de traitement n'est requis au MVP, un traitement manuel occasionnel par l'admin suffit à ce stade.

NOUVEAU v1.16

2.9 — Rétention et suppression des données

La Politique de Confidentialité (Article 9) définit déjà, par catégorie de donnée, une politique de rétention qualitative (compte inactif, photos de copies, historique de chat, données après clôture...). Cette section en fournit le mécanisme technique — sans lequel cette politique resterait une promesse sans base réelle, exactement le piège identifié dans le référentiel de sécurité ("pas juste un flag deleted=true qui laisse tout accessible en base").

2.9.1 — Cycle de vie d'un compte élève

Trois états successifs, matérialisés par `Eleve.statutCompte` (cf. §4.1) :

- ACTIF — état par défaut. `Eleve.derniereActiviteLe` est mis à jour à chaque connexion ou action significative (chat, correction, quiz).
- INACTIF_NOTIFIE — déclenché par un job BullMQ hebdomadaire qui détecte les élèves sans activité depuis un seuil défini (valeur de départ : 6 mois, non figée — même logique que les seuils du §2.2.2, à ajuster après observation réelle). Le parent (si un lien vérifié existe, via son canal préféré §2.2.3) est notifié ; à défaut de parent lié, l'élève lui-même est notifié par SMS. `dateNotificationInactivite` est enregistrée.
- ANONYMISE — déclenché soit automatiquement par un second job hebdomadaire si aucune activité n'a repris dans un délai de grâce de 60 jours après la notification, soit immédiatement sur demande explicite du parent (clôture de compte, cf. §2.2, CGU Article 7.9) sans attendre ce délai. Chaque transition d'état (détection d'inactivité, anonymisation automatique ou manuelle) est journalisée dans `AuditLogSecurite` (v1.23, cf. §4.6) — condition nécessaire pour que ce mécanisme constitue une preuve technique réelle, pas seulement un changement de valeur en base.

2.9.2 — Ce que fait l'anonymisation, concrètement

Un job BullMQ dédié exécute, de façon idempotente et irréversible :

- Suppression complète du contenu pédagogique identifiable : `TentativeEpreuve.photoUploadKeys` (objets R2 supprimés, pas seulement dé-référencés), `ConversationChat/MessageChat`, `Lacune`, `Quiz/QuizQuestion`, `SessionActivite`.
- Anonymisation (pas suppression) de la ligne `Eleve` elle-même — nom remplacé par une valeur générique, `pinHash` invalidé — plutôt qu'un hard delete, pour préserver l'intégrité référentielle avec `Abonnement/`

Païement, dont la durée légale de conservation comptable prime (cf. Politique de Confidentialité, ligne “Données de transaction et de facturation”).

- Suppression du ParentEleveLink associé.
- CorrectionDetail (notes/feedback, sans photo ni contenu identifiable) peut être conservé de façon anonymisée pour les besoins d'agrégats admin (§2.3), ou supprimé — à trancher selon la tolérance au risque souhaitée ; par défaut, suppression complète pour rester du côté prudent.

2.9.3 — Archivage des copies en fin d'année scolaire

Job annuel (déclenché à la rentrée scolaire, septembre, cf. §4.2.1 pour le calcul de l'année scolaire en cours) : les TentativeEpreuve.photoUploadKeys rattachées à des épreuves d'une anneeScolaire antérieure à l'année en cours moins un an sont supprimées de R2 (objets, pas seulement métadonnées) — CorrectionDetail (note, feedback) est conservé pour préserver l'historique de progression affiché au parent, cohérent avec “suppression ou archivage restreint” (Politique de Confidentialité).

2.9.4 — Ce qui n'est pas concerné par ces jobs

- OtpVerification — déjà à expiration courte et usage unique (§4.1), aucun job supplémentaire nécessaire.
- Paiement/Abonnement/WebhookLog — conservés pour la durée légale comptable, non anonymisés tant que cette durée n'est pas expirée.
- AuditLogSecurite — conservé pour une durée limitée mais distincte, dédiée à la détection d'incidents (cf. réf. sécurité §7) ; référence un utilisateurId qui peut pointer vers un compte déjà anonymisé, sans que ça pose de problème (l'anonymisation ne casse pas la traçabilité sécurité).

3. Stack technique recommandée

La stack retenue privilégie un monolithe Next.js bien structuré, cohérent avec les recommandations de scalabilité déjà validées (pas de microservices ni de Kubernetes à ce stade) et avec l'environnement de développement existant (Claude Code sur Windows, Docker).

Couche	Choix retenu	Justification
Framework fullstack	Next.js 15.5 (App Router) + TypeScript	Déjà en place sur Klarity ; SSR/API routes unifiées ; écosystème mature pour un solo-développeur.
Base de données	PostgreSQL 16 (conteneur Docker)	Scale largement à plusieurs dizaines de milliers d'utilisateurs actifs sans partitionnement (cf. section 8).
ORM	Prisma	Déjà utilisé sur les autres projets (Klarity/stage-platform) ; migrations versionnées, typage fort.
Authentification	Auth.js v5 (NextAuth), stratégie JWT stateless — access token court, refresh token long avec rotation (cf. §7)	Session sans état → scaling horizontal simple derrière un load balancer. Couvre les trois rôles (ADMIN/ PARENT/ELEVE) ; connexion élève par code + PIN (cf. §2.7), parent par code + téléphone + OTP (§2.2), admin par email + mot de passe + 2FA (§4.1).

Couche	Choix retenu	Justification
		Même politique de durée de vie des tokens pour les trois rôles, y compris l'élève (précision v1.17, cf. §2.7).
File d'attente / jobs asynchrones	BullMQ + Redis (conteneur Docker)	Standard Node.js le plus simple à opérer seul ; découple correction IA, génération de quiz, notifications SMS/ WhatsApp de la requête HTTP. Ses "repeatable jobs" (planification type cron intégrée à BullMQ) couvrent nativement le quiz quotidien (§4.3), le rappel de renouvellement d'abonnement (§5.5) et les jobs de détection d'inactivité/anonymisation (§2.9) — aucun outil de cron externe (node-cron, crontab système) n'est nécessaire.
Stockage fichiers	Cloudflare R2 (S3-compatible)	Déjà retenu dans le brief initial ; CDN natif, URLs signées à durée limitée pour les copies et corrigés.
IA — modèles de langage	API Anthropic Claude (Haiku + Sonnet, voir section 6)	Séparation coût/performance selon la criticité de la tâche.
Recherche vidéo	YouTube Data API v3	Alimente le pipeline automatisé du catalogue vidéo (cf. §2.5) : recherche de candidats par notion/matière/ classe, sans intervention admin.
Lecture vidéo	Intégration YouTube via <code><iframe></code> sur youtube-nocookie.com (mode confidentialité renforcée de l'IFrame Player API)	Aucun hébergement ni transcodage vidéo propre à Klarity : seules les métadonnées et l'ID YouTube sont stockés en base (cf. entité Vidéo, §4.2). Le domaine youtube-nocookie.com évite le dépôt de cookies de tracking tant que l'élève n'a pas interagi avec le lecteur — cohérent avec la contrainte "public mineur" (§1.2) et le principe de collecte minimale (réf. sécurité §1).
Paieement	NotchPay (Mobile Money uniquement), voir section 5	Agrégateur imposé par le contexte camerounais ; Stripe/PayPal non viables pour une entité enregistrée au Cameroun. Paiement par carte retiré du périmètre (cf. §2.4) — élimine toute exposition PCI-DSS côté Klarity.
Validation des entrées	Zod	Cohérent avec Next.js/Prisma ; validation stricte de toute route API (cf. sécurité §7).
Observabilité	Sentry (erreurs) + logs structurés (pino) + monitoring d'uptime	Détection immédiate d'un échec de correction, d'un webhook de paiement ou d'un timeout IA.

Couche	Choix retenu	Justification
Notifications	SMS OTP (fournisseur à sélectionner localement) + WhatsApp Business API (notifications sortantes uniquement, cf. §1.2, §2.2.3)	OTP obligatoire pour la connexion parent ; même canal réutilisé pour les rappels de renouvellement d'abonnement (§5.5), désormais envoyés aux deux destinataires disponibles (cf. §2.6) — pas de fournisseur supplémentaire à intégrer pour les OTP. WhatsApp Business API réintroduit v1.11, mais limité à l'envoi de messages-modèles sortants (résumés de progression, rappels) : aucun flux conversationnel entrant à gérer, à ne pas confondre avec le mode allégé interactif toujours repoussé en post-MVP (cf. §1.2, §10).
Génération de PDF	Librairie de rendu HTML → PDF côté serveur (ex. Puppeteer/Playwright headless, ou équivalent) [nouveau v1.11]	Alimente l'export du rapport mensuel parent (§2.2). Génération synchrone à la demande à partir des données déjà en base — pas de nouveau stockage R2 nécessaire, le PDF n'est pas persisté après téléchargement.
Conteneurisation / déploiement	Docker Compose (dev et prod), VPS ou Railway	Suffisant jusqu'à plusieurs milliers d'utilisateurs actifs ; pas de complexité d'orchestration prématurée.

3.1 — Services Docker Compose (développement)

```

services:
  app:      # Next.js (App Router) – API routes + rendu web
  worker:   # Process BullMQ dédié aux jobs asynchrones (correction, quiz,
notifications)
  postgres: # PostgreSQL 16 – base de données principale
  redis:    # Redis – file d'attente BullMQ + cache applicatif (vidéos/lacunes,
matières)
  # -- optionnel dev --
  adminer:  # Interface d'administration légère pour Postgres en local

```

Le service worker exécute le même code applicatif que app mais dans un process séparé dédié au traitement des files BullMQ (correction de copie, génération de quiz, envoi SMS/WhatsApp, rappel de renouvellement, détection d'inactivité et anonymisation, cf. §2.9) — c'est ce qui matérialise la séparation synchrone/asynchrone exigée en section 8.1.

4. Modèle de données (PostgreSQL / Prisma)

Le schéma ci-dessous couvre chaque élément fonctionnel des trois documents sources (brief produit, plan de scalabilité, référentiel de sécurité). Il est présenté par domaine, avec pour chaque entité ses champs clés, ses relations et les contraintes issues des exigences de sécurité/scalabilité déjà validées.

Mise à jour v1.27 — schéma finalisé. Le détail Prisma complet (types exacts, index, contraintes) a été généré intégralement dans `schema.prisma` à partir de la structure ci-dessous : **26 modèles, 22 enums**, couvrant tous les domaines §4.1 à §4.6 et tous les index prioritaires du §4.7. Fichier compagnon à ce document, à placer à la racine du projet avant `prisma migrate dev` (Phase 0, §10). Deux écarts assumés par rapport à des brouillons antérieurs à cette version : (1) le contenu pédagogique n'est **pas** normalisé en tables Module/Thème — il est porté par `ProgrammeOfficiel.contenuStructure` (Json), conformément à la décision du §4.2.3 ; (2) il n'existe **pas** de modèle `Classe / Serie` autonome — `NiveauClasse` et `Filiere` sont des enums Prisma portés directement par les entités qui en ont besoin (`Eleve`, `Epreuve`, `ProgrammeOfficiel`, `Matiere`).

4.1 — Domaine : Comptes et authentication

Entité	Champs clés	Relations / Contraintes
Eleve	id, nom, codeEleve (unique), classe (enum), filiere (enum: A/C/D/TI, nullable — 1ère/Terminale uniquement, jamais renseigné pour un élève de 3ème), langue (FR/EN), pinHash, pinTentativesEchouees (int, défaut 0), pinVerrouilleJusqua (timestamp, nullable), statutCompte (enum: ACTIF/INACTIF_NOTIFIE/ANONYMISE, défaut ACTIF), derniereActiviteLe (timestamp, nullable), dateNotificationInactivite (timestamp, nullable), dateAnonymisation (timestamp, nullable), createdAt	1—N vers TentativeEpreuve, Lacune, Quiz, ConversationChat, Abonnement, SessionActivite. Index sur codeEleve (recherche fréquente, cf. §4.7). pinHash ne stocke jamais le PIN en clair (même logique que OtpVerification.codeOtpHash) ; pinTentativesEchouees/ pinVerrouilleJusqua pilotent le rate limiting de connexion élève (cf. §2.7 et §7). filiere retypée v1.13 (était un binaire Générale/TI, incohérent avec l'affichage "Série D" déjà présent dans les maquettes depuis le début) : quatre séries réelles du système camerounais — A, C, D, TI — sélectionnées à plat à l'inscription, sans étape intermédiaire "générale puis série". Périmètre confirmé v1.13.1 : A/C/D/TI uniquement pour cette première version — décision assumée, pas une omission. D'autres séries existantes au Cameroun (E, F, G...) seront ajoutées ultérieurement ; l'enum est conçu pour absorber cet ajout par simple extension de valeurs, sans changement de structure ni migration lourde. Champs de rétention ajoutés v1.16 (§2.9) : statutCompte/ derniereActiviteLe/ dateNotificationInactivite/ dateAnonymisation pilotent le cycle de vie ACTIF → INACTIF_NOTIFIE → ANONYMISE, implémentant techniquement la politique de rétention déjà définie dans la Politique de Confidentialité (Article 9).
Parent		

Entité	Champs clés	Relations / Contraintes
	id, telephone (unique), createdAt, derniereConnexion, dernierEleveConsulteld (String, nullable, FK → Eleve)	N—N vers Eleve via ParentEleveLink (un parent peut suivre plusieurs enfants). dernierEleveConsulteld est mis à jour à chaque changement de sélection et pilote le sélecteur d'enfant côté dashboard (cf. §2.2).
ParentEleveLink	id, parentId (FK), eleveld (FK), codeUtilise, dateLiaison	Matérialise le lien vérifié parent ↔ élève ; le dashboard parent ne doit rien afficher tant que cette ligne n'existe pas (cf. réf. sécurité §1, §2).
NotificationPreference	id, parentId (FK, 1—1), canal (enum: SMS/WHATSAPP, défaut SMS), frequence (enum: HEBDOMADAIRE/ MENSUEL/ CRITIQUE_UNIQUEMENT), updatedAt	[nouveau v1.11] Pilote les résumés de progression (§2.2.3) — n'affecte ni l'OTP de connexion (toujours SMS) ni le rappel de renouvellement (§5.5, toujours envoyé indépendamment de cette préférence). Une ligne créée avec des valeurs par défaut à la première connexion du parent.
OtpVerification	id, telephone, codeOtpHash, expiration, utilise (bool), tentatives, createdAt	Jamais de code OTP en clair en base (hash uniquement) ; expiration courte, usage unique (cf. réf. sécurité §2).
Admin	id, email (unique), motDePasseHash, twoFactorSecret, role (enum)	2FA obligatoire (cf. réf. sécurité §2) ; role = SUPERADMIN ADMIN pour permettre une délégation future.

4.2 — Domaine : Contenu pédagogique

Entité	Champs clés	Relations / Contraintes
Matiere	id, nom, classesConcernees (enum[]), filiereRequise (enum[]: sous-ensemble de {A,C,D,TI}, nullable), banqueDisponible (bool)	Périmètre complet v1.7 : la table couvre toutes les matières du programme scolaire camerounais pour 3ème/1ère/Terniale, pour servir de sélecteur au chat généraliste (§2.1). Valeurs v1.25, cohérentes avec la table de référence du §2.1 : Maths, Français — communes aux 4 séries et à la 3ème, banqueDisponible=true ; PCT (Physique-Chimie-Technologie) — 3ème exclusivement (v1.25, matière combinée propre au 3ème, remplace "Physique" à cette classe ; Physique — 1ère/Terniale, filiereRequise={C,D,TI} (absente pour la série A) ; Chimie — 1ère/Terniale, filiereRequise={C,D,TI} ; SVT — 3ème/ 1ère/Terniale, filiereRequise={A} (n'a d'effet qu'en 1ère/Terniale ; accès inconditionnel en 3ème, où elle coexiste avec PCT) ; Philosophie —

Entité	Champs clés	Relations / Contraintes
		1ère/Terminale, filiereRequise={A} (réintroduite v1.22) ; Système d'Information, Programmation, Réseau — 1ère/Terminale, filiereRequise={TI} ; Anglais, Histoire-Géo — banqueDisponible=false (chat uniquement, cf. §1.2). Seules les matières banqueDisponible=true peuvent être référencées par Epreuve, ExempleCorrection, ProgrammeOfficiel (§4.2.3) et le mode chat contextualisé à une épreuve (§4.4). Pattern symétrique à noter : Physique et Chimie partagent exactement le même filiereRequise={C,D,TI} ; SVT et Philosophie partagent exactement le même filiereRequise={A}.
Epreuve	id, matiereId (FK), classe, filiere (enum: A/C/D/TI, nullable — même enum que Eleve.filiere, cf. v1.13), titre, anneeScolaire, fichePdfKey (R2), corrigeReferenceKey (R2, accès restreint), ajouteParAdminId (FK), createdAt	fichePdfKey/corrigeReferenceKey = clés d'objets R2, jamais d'URL publique stockée en dur (URL signée générée à la demande, cf. réf. sécurité §3). filiere confirmée v1.14 comme critère de filtrage réel : les épreuves diffèrent substantiellement d'une série à l'autre pour Maths, Physique, Français et Philosophie, pas seulement pour les matières TI. filiere reste null uniquement pour les épreuves de 3ème. La banque d'épreuves (§2.1) filtre désormais par classe + filiere + matiere. Précision v1.26 : classe est un différenciateur de contenu tout aussi réel que filiere, confirmé notamment par les matières TI, dont le programme officiel diffère entre 1ère et Terminale (cf. §4.2.3).
Video	id, titre, providerVideoId, matiereId (FK), notionAssociee, classe/filiere (nullable), langue, sourceAutomatisee (bool, toujours true)	providerVideoId = identifiant YouTube uniquement (pas de fichier vidéo stocké) ; lecture côté client via <code><iframe src="https://www.youtube-nocookie.com/embed/{providerVideoId}"></code> (cf. stack §3). notionAssociee sert de clé de correspondance avec Lacune.notion pour la recommandation automatique. Table alimentée exclusivement par le pipeline automatisé (§2.5) — aucune écriture manuelle admin prévue.
LacuneVideoCache		

Entité	Champs clés	Relations / Contraintes
	id, notionCle (unique), videoIdJsJson, dateExpiration	Cache applicatif Redis-backed prévu en section 8.5 — évite de re-consommer l'API YouTube/LLM à chaque requête identique.
DateExamen	id, typeExamen (enum: BEPC/PROBATOIRE/BAC), anneeScolaire (ex. "2026-2027"), dateExamen (date, nullable si pas encore annoncée), datePeriodeEstimee (text, nullable — ex. "courant juin 2027"), ajouteParAdminId (FK), updatedAt	Une ligne par type d'examen et par année scolaire, gérée exclusivement par l'admin (§2.3). dateExamen prime sur datePeriodeEstimee dès qu'elle est connue. Le compte à rebours affiché à l'élève est calculé côté serveur en résolvant automatiquement le bon typeExamen selon Eleve.classe et l'année scolaire en cours (cf. §4.7).

Mapping classe → typeExamen, dérivé directement de Eleve.classe : 3ème → BEPC, 1ère → Probatoire, Terminale → BAC. La filière (générale/TI) n'affecte pas le typeExamen retenu pour le compte à rebours.

4.2.1 — Calcul et affichage du compte à rebours

L'année scolaire en cours est déterminée dynamiquement côté serveur à partir de la date du jour (l'année scolaire camerounaise démarre en septembre) — elle n'a donc pas besoin d'être stockée par élève. Le serveur récupère la ligne DateExamen correspondant au typeExamen déduit de la classe de l'élève et à l'année scolaire en cours.

- Si dateExamen est renseignée : affichage d'un nombre de jours restants (ex. "BAC dans 87 jours") — simple différence de jours calendaires.
- Si seule datePeriodeEstimee est disponible : affichage de la période sans chiffre précis (ex. "BAC prévu courant juin 2027") — pour ne jamais induire l'élève ou le parent en erreur tant que la date n'est pas officielle.
- Côté admin (§2.3), le flux type est : en début d'année scolaire, saisie d'une estimation ; dès publication de la date officielle par le MINESEC, l'admin renseigne dateExamen, qui prend alors automatiquement le pas sur l'estimation.

4.2.2 — Exemples de correction (RAG / few-shot pour Français et Philosophie)

Problème identifié. Contrairement aux matières scientifiques (Maths, Physique, Chimie, PCT) — et, par défaut, SVT, dont la correction suit le même principe de vérification par barème plutôt qu'une évaluation méthodologique, sauf indication contraire à trancher plus tard —, la correction Français et Philosophie repose sur une évaluation méthodologique (structure de dissertation, qualité de l'argumentation) plutôt que sur une simple vérification de résultat. Sans contexte méthodologique injecté dans le prompt, la notation d'un modèle IA varie trop d'une correction à l'autre pour être fiable.

PRÉCISION v1.22. Philosophie, un temps retirée de la banque d'épreuves/correction (v1.19), y est revenue en v1.22 — réservée à la série A (cf. §2.1, §4.2). Le type DISSERTATION_PHILO ci-dessous est donc à nouveau actif, mais seulement pour les corrections d'épreuves de Philosophie de la série A.

typeExercice	Total	Répartition officielle	Contexte
DISSERTATION_PHILO	20 pts	Introduction 3 · Développement (thèse/	Philosophie — Baccalauréat & Probatoire

typeExercice	Total	Répartition officielle	Contexte
		antithèse/synthèse, 4 pts/ partie) 12 · Conclusion 3 · Présentation & langue 2	
DISSERTATION_LITTERAIRE	20 pts	Compréhension & problématique 3 · Organisation & plan 3 · Argumentation 8 · Conclusion 3 · Expression & présentation 3	Épreuve de Français
CONTRACTION_TEXTE	10 à 12 pts	Respect du volume 2 · Fidélité au texte 4 · Qualité de la reformulation 3 · Rédactique & enchaînement 2 à 3	Littérature — souvent couplé à DISCUSSION dans le même sujet
DISCUSSION	10 pts	Introduction 2 · Développement (thèse/ antithèse) 5 · Conclusion 1,5 · Langue & présentation 1,5	Littérature, Série A — “sujet 1” : contraction (10 pts) + discussion (10 pts) sur une idée du texte d'appui

Cas particulier — sujet combiné (Série A) : lorsqu'une épreuve associe contraction de texte et discussion dans le même sujet, le pipeline de correction résout les deux ExempleCorrection correspondants (CONTRACTION_TEXTE puis DISCUSSION) et évalue chaque partie séparément avant de combiner les deux notes partielles en une note globale — aucun champ supplémentaire requis sur TentativeEpreuve (§4.3), le détail par partie reste dans CorrectionDetail.pointsForts/pointsManques (json).

Entité	Champs clés	Relations / Contraintes
ExempleCorrection	id, matiereId (FK), typeExercice (enum: DISSERTATION_PHILO/ DISSERTATION_LITTERAIRE/ CONTRACTION_TEXTE/ DISCUSSION/ COMMENTAIRE_COMPOSE/ EXPRESSION_ECRITE/ CORRECTION_ORTHOGRAPHIQUE), enonceModele (text), baremeStructure (json ou text), exempleReponseModele (text), notesMethodologiques (text), langue, ajouteParAdminId (FK), createdAt	Contenu de référence (les 7 barèmes officiels ci-dessus + exemples de réponses modèles) chargé en base au démarrage du projet, géré comme donnée de configuration versionnée, pas comme contenu élève. Confirmé v1.15 : pas de champ filiere ici, contrairement à Epreuve (§4.3, v1.14) — le barème méthodologique officiel est identique quelle que soit la série de l'élève ; seul le contenu des épreuves elles-mêmes varie par classe et par série.

Mécanisme : pour toute correction (AIProvider.corrigerCopie(), cf. §6.2) portant sur une matière méthodologique (Français, Philosophie — série A uniquement pour cette dernière), le pipeline de correction récupère le ou les ExempleCorrection correspondant à matiereId et typeExercice, et les injecte comme exemples few-shot dans le prompt envoyé à Claude Sonnet, en plus du barème structuré de l'épreuve elle-même. Pour les matières scientifiques (dont Chimie et PCT) et pour SVT (par défaut), ce mécanisme est ignoré — seul le barème de l'épreuve (Epreuve.corrigeReferenceKey, §4.2) est utilisé.

Le barème et les exemples de ExempleCorrection sont réinjectés à chaque appel de correction pour une même matière/type d'exercice — candidats naturels au prompt caching déjà identifié dans le plan de scalabilité (§8), pour maîtriser le coût par appel Sonnet malgré un prompt plus long.

NOUVEAU v1.24

4.2.3 — Programmes officiels (référentiel de contenu par matière/classe/série)

Trou comblé. Le document supposait déjà l'existence d'un “programme officiel” à plusieurs endroits (§6.4 : prompt caching du “barème et du programme officiel” à chaque correction ; §6.2 : AIProvider.chat() reçoit un contexteMatiere décrit comme “programme scolaire de la matière” ; §7 : le chat doit refuser toute question “hors du programme scolaire”) — sans jamais définir où ce contenu est stocké. Cette section comble ce vide.

Entité	Champs clés	Relations / Contraintes
ProgrammeOfficiel	id, matiereId (FK), classe (enum), filiere (enum: A/C/D/TI, nullable — même logique que Epreuve.filiere, §4.3), contenuStructure (json — tableau de modules, chacun avec un titre et une liste de notions), fichierSourceKey (R2, nullable — document source original, ex. un .docx de programme OBC), versionSource (text, ex. “OBC 2026”), langue, ajouteParAdminId (FK), createdAt, updatedAt	Une ligne par couple (matière, classe, série) — toujours distinguée par classe, pas seulement par série (précision v1.26) : ex. Maths Terminale C ≠ Maths Terminale D (série), mais aussi Système d'Information 1ère TI ≠ Système d'Information Terminale TI (classe). filiere reste null pour les matières communes à toutes les séries d'une classe (ex. Français) ou pour la 3ème (pas de filière) ; classe, elle, n'est jamais optionnelle. Double usage : (1) contenuStructure alimente le contexte système des appels IA (chat §6.2, correction §6.4 — prompt caching) ; (2) les notions listées dans contenuStructure deviennent la taxonomie de référence pour Lacune.notion, pointsManques[], notion et Video.notionAssociee (§4.3, §2.5). fichierSourceKey permet de conserver le document d'origine pour référence admin, sans que ce soit la donnée réellement consommée par l'IA — c'est contenuStructure qui l'est.

Gestion : alimentée par l'admin (§2.3) au démarrage du projet pour chaque matière/classe/série du périmètre MVP (cf. table de référence §2.1), à partir des documents de programme officiels déjà rassemblés — donnée de configuration versionnée, comme ExempleCorrection, pas du contenu élève.

Précision v1.27 / révision v1.29 — périmètre exact. Le périmètre MVP comptait **9 couples** au sens du référentiel de programmes (une entrée par classe/série : la 3ème + Première × {A, C, D, TI} + Terminale × {A, C, D, TI} = 1 + 4 + 4), pour 9 fichiers JSON. La révision v1.29 ajoute SVT à la banque d'épreuves et à la correction pour les séries C, D et TI en 1ère et Terminale (§2.1, §4.2) : **6 couples matière/classe/série supplémentaires, soit 15 au total**. Les 6 sections svt correspondantes sont déjà présentes dans les fichiers de programmes C/D/TI et sont chargées dans contenuStructure — SVT n'ajoute pas de fichier JSON, seulement une section aux fichiers C/D/TI existants. Le Français reste au socle de chaque série (obligatoire à l'examen national) : cet ajout est un complément, jamais un remplacement. La 3ème continue de compter comme un périmètre à part entière.

4.3 — Domaine : Suivi pédagogique de l'élève

Entité	Champs clés	Relations / Contraintes
TentativeEpreuve	id, eleveld (FK), epreuveId (FK), numeroTentative, photoUploadKeys (json — tableau ordonné de clés R2, une par page photographiée), statut (EN_ATTENTE/EN_TRAITEMENT/TERMINE/ERREUR), dateSoumission, dateTraitement	Une ligne par upload ; numeroTentative permet le retraitement illimité prévu au cahier des charges. Statut piloté par le job BullMQ (cf. §8.1). Seule la première tentative (numeroTentative = 1) déclenche un appel Sonnet réel ; les tentatives suivantes sont acceptées mais renvoient la même correction, sans nouveau traitement IA (cf. §2.1, §6.4). Champ note retiré v1.23 — redondant avec CorrectionDetail.note/ noteOverride (source de vérité unique depuis v1.1). Upload multi-pages v1.10 : photoUploadKey (unique) devient photoUploadKeys, un tableau de clés R2 dans l'ordre de saisie par l'élève. L'ordre du tableau fait foi pour la lecture séquentielle par l'IA (§6.2).
CorrectionDetail	id, epreuveId (FK) + eleveld (FK), pointsForts (json), pointsManques (json — tableau structuré d'objets {notion, detail}), feedbackDetaillé (text), modeleIA (= 'claude-sonnet'), tokensInput, tokensOutput, signalee (bool, défaut false), motifSignalement (enum: LECTURE_ILLISIBLE/ BAREME_INCORRECT/AUTRE, nullable), commentaireEleve (text, nullable), noteOverride (nullable), justificationOverride (text, nullable), overrideParAdminId (FK, nullable), dateSignalement (nullable), dateTraitementSignalement (nullable), createdAt	Relation corrigée v1.1 : une seule CorrectionDetail par couple (élève, épreuve), et non plus une par tentative — cohérent avec la règle "la première correction fait foi" (§2.1). tokensInput/ Output alimentent directement UsageIA (§4.6). Champs de signalement ajoutés v1.2 (§2.8) : noteOverride prime sur la note calculée par l'IA. pointsManques structuré v1.5 : chaque entrée porte un champ notion utilisant le même vocabulaire que Lacune.notion.
Lacune	id, eleveld (FK), matiereId (FK), notion, niveauMaitrise (int, 0–100), sourceTentativeId (FK, nullable), dateDetection, dateMiseAJour, resolu (bool)	Index sur eleveld — lecture très fréquente pour quiz journalier et dashboard parent. Création v1.5 : une ligne est créée (ou mise à jour si elle existe déjà pour ce couple élève/ notion) immédiatement à chaque correction, à partir de CorrectionDetail.pointsManques. niveauMaitrise retypé v1.12 : calcul déterministe, sans appel IA — ratio (réponses correctes récentes / total de réponses observées) sur cette notion, recalculé à chaque nouvelle donnée. resolu passe à true quand

Entité	Champs clés	Relations / Contraintes
		niveauMaitrise dépasse un seuil (ex. 80%) sur un nombre minimum de réponses récentes. Les couleurs/ badges affichés sont dérivés de niveauMaitrise côté frontend, pas stockés séparément.
Quiz	id, eleveId (FK), matiereId (FK), dateGeneration, statut, score, origine (enum: JOURNALIER/CIBLE), lacuneCibleId (FK → Lacune, nullable)	Génération quotidienne planifiée (cron BullMQ) pour origine=JOURNALIER, ciblée sur les Lacune.resolu = false de l'élève. origine=CIBLE (v1.11) : généré à la demande depuis l'écran "Mes lacunes", lacuneCibleId renseigné, questions portant exclusivement sur cette notion — même AIProvider.genererQuiz() que le quiz journalier (§6.2), simplement invoqué de façon synchrone/à la demande plutôt que par le cron.
QuizQuestion	id, quizId (FK), lacuneId (FK, nullable), enonce, choixJson, bonneReponse, reponseEleve, correcte	lacuneId permet de mesurer la résolution effective d'une lacune après plusieurs quiz.
SessionActive	id, eleveId (FK), dateDebut, dateFin, dureeSecondes, canal (enum: WEB, WHATSAPP réservé pour extension post-MVP — cf. §1.2)	Alimente le "temps passé par jour" affiché au parent ; agrégée côté requête, jamais recalculée en tâche de fond lourde.

4.4 — Domaine : Chat-tuteur IA

Entité	Champs clés	Relations / Contraintes
ConversationChat	id, eleveId (FK), matiereId (FK, obligatoire), epreuveId (FK, nullable), titre, createdAt	Regroupe les échanges par sujet pour permettre l'historique côté élève. Deux cas valides, confirmés v1.4 — epreuveId NULL : chat par matière (mode 1, §2.1) ; epreuveId renseigné : chat contextualisé à l'épreuve (mode 2, §2.1). matiereId reste obligatoire dans les deux cas — c'est epreuveId qui distingue le mode.
MessageChat	id, conversationId (FK), role (ELEVE/ ASSISTANT), contenu (text), modeleIA (= 'claude-haiku'), tokensInput, tokensOutput, createdAt	contenu élève : sanitized avant stockage (cf. réf. sécurité §6) ; jamais interprété comme instruction système côté prompt (cf. réf. sécurité §3).

4.5 — Domaine : Abonnement et paiement

Entité	Champs clés	Relations / Contraintes
Abonnement		

Entité	Champs clés	Relations / Contraintes
	id, eleveId (FK), plan (GRATUIT/PREMIUM), dateDebut, dateFin, statut (enum), prixApplique, renouvellementAuto (bool), dateProchainRenouvellement (date), rappelEnvoye (bool, défaut false)	prixApplique fige le montant réellement facturé, y compris en période de réduction saisonnière — ne jamais recalculer a posteriori depuis un tarif courant. dateProchainRenouvellement et rappelEnvoye pilotent le rappel automatique (cf. §5.5) ; rappelEnvoye est remis à false à chaque nouveau cycle de facturation.
Paieiment	id, abonnementId (FK), payeurTelephone, payeurRole (enum: PARENT/ELEVE), montant, devise (XAF), methode (MOBILE_MONEY), statut (EN_ATTENTE/REUSSI/ECHEC/REMBOURSE), referenceTransaction, idempotencyKey (unique), datePaieiment	idempotencyKey garantit qu'un même événement webhook ne peut créditer l'abonnement qu'une seule fois (cf. réf. sécurité §4 et plan de scalabilité §7). payeurRole ajouté v1.1 : distingue si le paiement a été initié depuis l'espace parent ou l'espace élève (cf. §2.6). methode réduit à une seule valeur v1.6 (paiement carte retiré, cf. §2.4).
WebhookLog	id, provider (ex. 'NOTCHPAY'), payloadBrut (json), signatureValide (bool), traitementStatut, createdAt	Journal immuable (append-only) de tout webhook reçu, y compris ceux rejetés pour signature invalide — sert de piste d'audit pour les litiges Mobile Money.

4.6 — Domaine : Monitoring, coûts IA et sécurité

Entité	Champs clés	Relations / Contraintes
UsageIA	id, eleveId (FK, nullable), typeUsage (enum: CHAT/QUIZ/CORRECTION/VIDEO_FILTRAGE), modele (haiku/sonnet), tokensInput, tokensOutput, coutEstime, date	Alimente le monitoring de coût par utilisateur exigé au plan de scalabilité §8. VIDEO_FILTRAGE ajouté v1.23. eleveId devient nullable pour ce typeUsage : le filtrage se fait par notion, potentiellement mutualisé entre plusieurs élèves via le cache (LacuneVideoCache).
AuditLogSecurite	id, typeEvenement (enum: LOGIN_FAIL/OTP_FAIL/PIN_FAIL/IDOR_BLOCKED/WEBHOOK_INVALID/COMPTE_INACTIF_DETECTE/COMPTE_ANONYMISE_AUTO/COMPTE_ANONYMISE_MANUEL/...), utilisateurId (nullable), ip (nullable), details (json), createdAt	Alimente les alertes prévues en réf. sécurité §7. PIN_FAIL ajouté v1.1. Événements de cycle de vie du compte ajoutés v1.23. COMPTE_ANONYMISE_MANUEL journalise en plus l'identité du parent à l'origine de la demande (dans details). ip devient nullable.

4.7 — Index prioritaires à la conception (cf. plan de scalabilité §3)

- Eleve.codeEleve — recherché à chaque tentative de connexion parent et élève.

- `Lacune.eleveld` — lu à chaque génération de quiz journalier et à chaque chargement du dashboard parent ; désormais aussi en écriture à chaque correction, volumétrie d'écriture à surveiller si le volume de corrections quotidiennes devient important.
- `Paiement.statut` et `Paiement.idempotencyKey` — critiques pour la fiabilité du traitement webhook.
- `TentativeEpreuve.eleveld` et `TentativeEpreuve.statut` — pour l'historique élève et le suivi de la file de traitement.
- Index composite `Epreuve(classe, filiere, matiereId)` — lu à chaque chargement de la banque d'épreuves (§2.1) ; filtrage désormais réel par série pour 1ère/Terminale.
- Index composite unique `DateExamen(typeExamen, anneeScolaire)` — lu à chaque chargement du dashboard élève/parent pour le compte à rebours.
- Index composite `Abonnement(dateProchainRenouvellement, rappelEnvoye)` — lu quotidiennement par le job de rappel de renouvellement (§5.5).
- Index composite `ExempleCorrection(matiereId, typeExercice)` — lu à chaque correction Français/Philosophie (§4.2.2).
- Index composite unique `ProgrammeOfficiel(matiereId, classe, filiere)` — lu à chaque appel de chat et de correction (§4.2.3) ; table de faible volumétrie (9 lignes au périmètre MVP, cf. §4.2.3 v1.27), même profil que `ExempleCorrection`.
- Index composite unique `CorrectionDetail(epreuveId, eleveld)` — garantit une correction unique par couple élève/épreuve (cf. §4.3).
- Index `CorrectionDetail(signalee, dateTraitementSignalement)` — lu par la file de revue admin (§2.3, §2.8).
- Index composite `Eleve(statutCompte, derniereActiviteLe)` — lu par les jobs hebdomadaires de détection d'inactivité et d'anonymisation (§2.9).
- Pagination systématique sur toute liste exposée (historique de corrections, quiz, élèves côté admin) — jamais de `SELECT *` sans limite.

5. Intégration paiement — NotchPay

Contexte. NotchPay est désormais implémenté (§5.1) — cette section documente l'architecture réelle et ce qui reste à valider avec de vraies clés sandbox ; aucune réécriture n'est nécessaire au passage en argent réel — seulement un changement de clés (test → live).

5.1 — Principe : abstraction du fournisseur de paiement

Toute la logique métier (création d'abonnement, mise à jour de statut, crédit du compte) s'appuie sur une interface `PaymentProvider` unique, jamais sur un appel direct au SDK/API NotchPay. Cela isole 100% du code applicatif d'un changement d'agrégateur ou d'un passage sandbox → live.

```
interface PaymentProvider {
  initierPaielement(montant, devise, methode, payeur): Promise<PaielementSession>
  verifierSignatureWebhook(payloadBrut, signatureRecue): boolean
  traiterWebhook(payloadBrut): Promise<ResultatPaielement>
}
// Implémentations :
// - MockPaymentProvider → simule un paiement réussi/échoué en local (dev sans accès
NotchPay)
// - NotchPayProvider → implémentation réelle (§5.3)
// Pas de sandbox/live séparé : une seule API NotchPay pour
les deux, seul le préfixe de clé (pk_test_/pk_live_) change.
// Sélection au démarrage via variable d'environnement :
PAYMENT_MODE = mock | notchpay
```

5.2 — Déjà construit, avant même les clés sandbox

- Le modèle de données complet (Abonnement, Paiement, WebhookLog) et toute la logique d'idempotence — indépendants du fournisseur réel.
- La page de paiement (saisie du numéro Mobile Money), accessible depuis l'espace parent et l'espace élève (cf. §2.6), et le pop-up de confirmation, branchés sur MockPaymentProvider qui simule les statuts REUSSI/ECHEC après un court délai, pour valider l'UX de bout en bout.
- Le handler de webhook, avec vérification de signature HMAC déjà codée mais désactivable en mode mock — pour être prêt dès réception des identifiants sandbox sans retravailler l'endpoint.
- Les tests d'idempotence (rejouer deux fois le même événement webhook mock) et de non-double-crédit d'abonnement.

5.3 — Ce qui reste à valider avec de vraies clés NotchPay

- Documentation officielle NotchPay récupérée : format exact du payload webhook et algorithme de signature (HMAC-SHA256, header x-notch-signature) — détail complet au journal v1.31 → v1.32.
- NotchPayProvider implémenté (§5.1) contre la documentation publique — premier paiement réel avec les clés sandbox de l'utilisateur à exécuter pour confirmer le comportement en conditions réelles.
- Valider en sandbox le scénario complet : Mobile Money réussi, Mobile Money échoué, webhook dupliqué (retry réseau).
- Vérifier si NotchPay fournit une liste d'IPs sources autorisées pour ses webhooks (référentiel sécurité §4) — si oui, ajouter un filtrage IP en plus de la vérification de signature HMAC (défense en profondeur, pas un remplacement) ; si non, la signature HMAC reste la protection principale, déjà couverte au §5.4.

5.4 — Rappel sécurité applicable dès l'implémentation réelle

- Vérification de signature HMAC obligatoire sur chaque webhook — aucun payload non signé n'est traité, même en cas de doute.
- Idempotence stricte via Paiement.idempotencyKey avant tout crédit d'abonnement.
- Aucune saisie ni transit de donnée de carte bancaire côté Klarity — le paiement carte est retiré du périmètre (cf. §2.4) ; seul le flux Mobile Money, initié par NotchPay, est utilisé.
- Journalisation immuable de chaque webhook reçu, traité ou rejeté, dans WebhookLog.

5.5 — Rappel de renouvellement automatique

Contexte. Le Mobile Money ne permet pas de prélèvement automatique silencieux — chaque paiement doit être confirmé activement par l'utilisateur. Le système doit donc rappeler proactivement l'échéance, jamais compter sur un renouvellement passif.

5.5.1 — Mécanisme

- Un job BullMQ planifié (cron quotidien) parcourt chaque jour les Abonnement actifs dont `dateProchainRenouvellement` tombe dans J-3 (3 jours avant échéance) et `rappelEnvoye` = false.
- Pour chaque abonnement concerné : envoi d'une notification avec un lien de paiement à un clic reprenant le tarif applicable — déterminé via `determinerPeriodeTarifaire()` à la date du renouvellement, pas celle de la souscription initiale (cf. §2.4.1). La notification est envoyée à tous les destinataires disponibles pour cet élève : le parent (si un `ParentEleveLink` vérifié existe, sur le canal SMS ou WhatsApp choisi dans `NotificationPreference`, cf. §2.2.3) et l'élève lui-même (toujours par SMS), plutôt qu'à un seul canal.
- `rappelEnvoye` passe à true après envoi (aux deux destinataires le cas échéant), pour ne jamais notifier deux fois pour la même échéance.
- Période de grâce : si le paiement n'est pas reçu à `dateProchainRenouvellement` + 3 jours, le statut `Abonnement.statut` bascule de ACTIF à EXPIRE, coupant l'accès premium — l'élève repasse automatiquement sur le plan GRATUIT.
- Un second job (même cron ou déclenché en cascade) applique cette bascule de statut de façon idempotente.

5.5.2 — Cohérence avec l'architecture déjà posée

Ce job utilise le même worker BullMQ que le reste (§3, §8) — aucune nouvelle brique d'infrastructure. Le canal de notification réutilise le fournisseur SMS déjà prévu pour l'OTP (§2.2). Ces ajouts couvrent les exigences posées : connaissance de la date pour proposer le bon plan (§2.4.1), mémorisation fiable du paiement (`prixApplique` déjà figé, §4.5), identification du payeur (`payeurRole`, §4.5), et rappel automatique multi-destinataire quelques jours avant échéance — le tout sans ajouter de nouvelle brique d'infrastructure, juste un job planifié de plus sur le worker déjà prévu.

6. Intégration IA — API Claude (accès API en attente)

Contexte. La clé API Anthropic n'est pas encore disponible. Comme pour NotchPay, l'architecture prévoit une abstraction du fournisseur de modèle afin que le développement (prompts, gestion de la file IA, UI de correction et de chat) avance dès maintenant sur un mode simulé, sans réécriture au moment de brancher la vraie clé.

6.1 — Répartition des modèles par usage

Usage	Modèle	Justification
Chat-tuteur (réponses aux questions de cours, contextualisé à l'épreuve en cours — cf. §2.1)	Claude Haiku	Volume élevé, latence attendue faible côté élève, coût à maîtriser sur un usage "illimité" en premium.
	Claude Haiku	

Usage	Modèle	Justification
Génération de quiz / exercices journaliers		Tâche structurée et répétitive, forte fréquence d'appel (quotidienne, par élève).
Correction d'épreuve (vision sur copie photographiée) — génère aussi les Lacune correspondantes à partir de pointsManques, sans appel IA distinct	Claude Sonnet	Tâche à plus fort enjeu : lecture d'une copie manuscrite, évaluation fine par rapport au barème, impact direct sur la note affichée à l'élève. Appel déclenché une seule fois par couple élève/épreuve (première tentative uniquement, cf. §2.1 et §4.3).
Affinage des lacunes (analyse croisée de l'historique)	Aucun — calcul déterministe	Simplification v1.12 : niveauMaitrise passe d'un enum à un pourcentage calculé (§4.3), sans interprétation qualitative — l'appel Sonnet précédemment prévu pour cette étape est retiré.

6.2 — Principe : service IA isolé (déjà amorcé côté produit)

Le brief initial prévoyait déjà une fonction unifiée `appelerModeleIA()`. Elle est étendue ici en une interface `AIProvider` avec sélection automatique du modèle selon le type de tâche, pour que l'appelant (route API, job BullMQ) n'ait jamais à connaître le nom du modèle utilisé.

```

interface AIProvider {
  chat(messages, contexteMatiere, contexteEpreuve?): Promise<ReponseIA> // -> route vers Haiku
  // contexteMatiere = contenuStructure de ProgrammeOfficiel (§4.2.3) pour la matiere/
  // classe/serie
  //           de l'eleve (toujours fourni ; couvre toute matiere du programme,
  //           y compris hors banque d'epreuves, cf. §1.2, §4.2)
  // contexteEpreuve = enonce + corrige de l'epreuve (fourni uniquement en mode
  //           chat contextualise, cf. §2.1, §4.4)

  genererQuiz(lacunes, matiere): Promise<QuizGenere> // -> route vers Haiku

  corrigerCopie(imageKeys, epreuveRef, bareme, exemplesFewShot?): Promise<Correction> //
  -> route vers Sonnet
  // exemplesFewShot : injecte les ExempleCorrection (§4.2.2) pour Francais/Philo,
  //           ignore pour les matieres scientifiques (bareme d'epreuve seul
  //           suffit)
  // N'est appelee que pour numeroTentative = 1 (cf. §4.3) - retraitements ulterieurs
  // lus directement depuis CorrectionDetail existant, sans appel IA.
  // Correction.pointsManques structure {notion, detail} (cf. §4.3) sert directement
  // a creer/mettre a jour les Lacune correspondantes - pas d'appel IA separe.
  // imageKeys : tableau ordonne (multi-pages, cf. §4.3 photoUploadKeys) - toutes les
  // pages sont transmises en une seule requete vision Sonnet, dans l'ordre.

  // affinerLacunes() retiree v1.12 : niveauMaitrise/resolu sont desormais calcules
  // de facon deterministe en code applicatif (ratio correct/total, cf. §4.3), sans appel
  IA.
}
// Implémentations :
// - MockAIProvider → réponses simulées réalistes (dev/tests sans clé API)
// - ClaudeAIProvider → appel réel à l'API Anthropic, modèle choisi selon la méthode
//   appelée
AI_MODE = mock | live
ANTHROPIC_API_KEY = <à renseigner dès obtention>

```

La résolution de exemplesFewShot (requête ExempleCorrection par matiereId + typeExercice) est faite en amont de l'appel, dans le job BullMQ de correction (§8.1) — l'interface AIProvider reste agnostique du contenu pédagogique, elle reçoit simplement des exemples déjà résolus.

6.3 — Ce qui peut être construit dès maintenant, sans clé API

- Toute la file BullMQ dédiée aux appels IA (retry avec backoff sur 429, débit contrôlé) — testable avec MockAIProvider qui peut simuler une erreur 429 à la demande.
- L'UI complète du chat contextualisé à l'épreuve, de l'upload de copie et de l'affichage de correction, alimentée par des réponses simulées réalistes.
- La structure exacte des prompts (séparation contexte système / contenu utilisateur, injection du barème et du programme avec prompt caching prévu, cadrage strict au contenu scolaire pour le chat — cf. §7), rédigée et versionnée dès maintenant, prête à être testée dès la clé obtenue.
- Le calcul et l'enregistrement de UsageIA (tokens, coût estimé) sur la base de tokens simulés, pour valider le tableau de bord de contrôle de coût avant même le premier appel réel.

6.4 — Contrôle des coûts (risque business identifié)

Un abonnement premium à 3 000-5 000 FCFA/mois avec usage IA présenté comme “illimité” peut devenir déficitaire sur les gros utilisateurs si le coût d’API dépasse la marge. À prévoir dès le MVP :

- Plafond technique haut mais réel même en premium (ex. nombre de messages chat/jour, nombre de corrections/jour) — jamais un “illimité” au sens littéral.
- Enregistrement systématique de UsageIA par élève, avec alerte admin sur les profils aberrants.
- Prompt caching pour le barème et le programme officiel (ProgrammeOfficiel.contenuStructure, cf. §4.2.3) réinjectés à chaque appel de correction — réduit le coût par appel Sonnet de façon significative.
- Correction unique par couple élève/épreuve (cf. §2.1, §4.3) : les tentatives de retraitement au-delà de la première ne déclenchent aucun nouvel appel Sonnet — levier de contrôle de coût significatif, la correction/vision étant l’appel le plus onéreux du système.

6.5 — À faire dès l’obtention de la clé API

- Implémenter ClaudeAIProvider en réutilisant les prompts déjà rédigés et versionnés.
- Valider en conditions réelles : temps de réponse Haiku sur le chat, qualité de correction Sonnet sur un échantillon de copies réelles, coût réel par appel comparé à l’estimation faite en mode mock.
- Basculer AI_MODE=live matière par matière si besoin, pour limiter le risque en cas d’écart de qualité constaté sur une matière donnée.

7. Sécurité — synthèse des exigences

Cette section consolide le référentiel de sécurité déjà produit pour Klarity. Le détail complet (checklist par domaine, schéma des flux critiques) reste disponible dans le document dédié ; il est résumé ici pour rester dans un cahier des charges unique.

Domaine	Exigences non négociables
Données de mineurs	Collecte minimale, dashboard parent verrouillé tant que le lien parent ↔ élève n’est pas vérifié, chiffrement au repos des données scolaires, droit de suppression effectif via un mécanisme réel d’anonymisation (cf. §2.9) — pas un simple flag, conformité Loi n°2000/011/2024/017.
Chiffrement en transit (nouveau v1.17)	TLS obligatoire sur tous les échanges — trafic client ↔ serveur (HTTPS strict, pas de fallback HTTP), et entre services internes si l’architecture venait à se séparer au-delà du monolithe app/worker actuel (§3, §8). S’ajoute au chiffrement au repos déjà prévu : les deux sont non négociables.
Authentification	Code élève généré par source d’aléa cryptographique, PIN élève haché en base avec rate limiting dédié (cf. §2.7, §4.1), rate limiting IP + téléphone sur la connexion parent, OTP à usage unique et expiration courte, JWT à durée de vie courte avec refresh rotatif, 2FA obligatoire pour l’admin.
Upload / vision IA	Validation du type MIME réel, taille max, re-encodage serveur — appliqués individuellement à chaque page uploadée (cf. §4.3) — avant stockage, URLs R2 signées à

Domaine	Exigences non négociables
	durée limitée par page, séparation stricte contexte système / contenu utilisateur dans les prompts.
Chat IA (nouveau v1.1, périmètre étendu v1.7)	Prompt système contraignant le chat au strict programme scolaire de la classe/filière de l'élève (cf. §2.1, §4.4), construit à partir de ProgrammeOfficiel.contenuStructure (§4.2.3) — périmètre plus large que la banque d'épreuves, mais toujours borné au scolaire : refus poli et redirection automatique en cas de question hors-programme, journalisation des dérives répétées, rate limiting par utilisateur sur les endpoints IA.
Paieement	Signature HMAC vérifiée sur chaque webhook, idempotence stricte, aucune donnée de carte bancaire côté Klarity (paiement carte retiré du périmètre, cf. §2.4), journalisation immuable des transactions.
Autorisation (IDOR)	Rôles ADMIN/PARENT/ELEVE appliqués côté middleware serveur, vérification d'appartenance de la ressource à chaque requête (jamais l'authentification seule), tests IDOR systématiques avant mise en production.
Validation & API	Zod sur toute entrée API, rate limiting par utilisateur sur les endpoints IA, headers de sécurité standards (CSP, etc.).
Observabilité sécurité	Alertes sur accès non autorisé détecté, anomalies de connexion, pics d'appels IA suspects, échecs de vérification de signature webhook.

Référence complète : *Klarity_Securite_Reference.md* — à conserver comme document compagnon, notamment pour le schéma détaillé des 5 flux critiques (inscription, connexion parent, upload/correction, paiement, accès aux données).

8. Scalabilité — synthèse des exigences

Principe	Application concrète sur Klarity
Synchrone / asynchrone	Upload de copie → réponse HTTP immédiate, traitement réel (correction, quiz, vidéo, SMS/WhatsApp, rétention/anonymisation) délégué à un worker BullMQ ; résultat récupéré par polling ou websocket.
IA comme service isolé	Retry avec backoff sur 429, file dédiée aux appels IA à débit contrôlé, abstraction provider (cf. section 6) pour changer de fournisseur par simple config.
Base de données	Pool de connexions Prisma dimensionné au nombre d'instances, index sur toutes les clés étrangères et champs de recherche fréquents (§4.7), pagination systématique, pas de partitionnement prématuré.
Stateless par design	Aucun état en mémoire du process Next.js ; sessions JWT ou en base, jamais en RAM locale — permet plusieurs instances derrière un load balancer sans affinité de session.

Principe	Application concrète sur Klarity
Cache multi-niveaux	Cache vidéo/lacune, prompt caching (barème et programme), CDN pour assets statiques et R2, Redis pour les données lues souvent et modifiées rarement (matières/chapitres).
Observabilité	Logging structuré (pas de console.log), Sentry, monitoring d'uptime sur auth/paiement/correction.
Idempotence financière	Chaque traitement de webhook NotchPay vérifie l'absence de traitement préalable avant de créditer un abonnement.
Contrôle des coûts IA	Plafonds techniques réels même en premium, correction unique par élève/épreuve (§6.4), monitoring de coût par utilisateur (§6.4).
À ne pas faire maintenant	Pas de microservices, pas de Kubernetes, pas de sharding — un monolithe Next.js + Docker Compose suffit jusqu'à plusieurs milliers d'utilisateurs actifs.

9. Architecture globale (vue d'ensemble)



10. Roadmap de mise en œuvre

Séquencement pensé pour ne jamais être bloqué par l'absence de clé API Claude, seule dépendance encore réellement en attente — NotchPay (§5) est désormais implémenté, bascule en argent réel par un simple changement de clés.

Phase	Contenu	Dépendances externes
Phase 0 — Socle	Docker Compose (app, worker, postgres, redis), prisma validate puis prisma migrate dev sur le schéma finalisé v1.27 (§4) , Auth.js v5 avec flux code + OTP côté parent et code + PIN côté élève (§2.7), rôles ADMIN/PARENT/ELEVE.	Aucune — développable immédiatement.
Phase 1 — Cœur pédagogique (mock IA)	Banque d'épreuves (périmètre MVP dépendant de la classe et de la série, cf. table de référence §2.1), upload copie multi-pages (§4.3), chargement des 9 programmes officiels par matière/classe/série (ProgrammeOfficiel, §4.2.3, périmètre confirmé v1.27) , pipeline correction/quiz branché sur MockAIProvider, chat généraliste contextualisé à l'épreuve (§2.1) en mock, détection immédiate de lacunes (§2.1, §4.3), dashboard parent avec alertes intelligentes et export PDF (§2.2.2, §2.2), préférences de notification (§2.2.3), quiz ciblé à la demande (§4.3), back-office admin (hors CA réel).	Aucune — MockAIProvider suffit pour valider tout le parcours.
Phase 2 — Paiement (mock puis sandbox)	Page de paiement accessible depuis l'espace parent et l'espace élève (§2.6), Abonnement/Paiement/WebhookLog, idempotence, MockPaymentProvider puis NotchPayProvider (implémenté, §5.3), rappel de renouvellement multi-destinataire (§5.5).	Clés sandbox NotchPay de l'utilisateur pour la première validation réelle — non bloquant, code déjà écrit (§5.3).
Phase 3 — Bascule IA réelle	Implémentation ClaudeAIProvider (Haiku + Sonnet), validation qualité sur échantillon réel, activation du monitoring de coût (UsageIA) en conditions réelles.	Clé API Anthropic Claude requise.
Phase 4 — Bascule paiement live	Bascule des clés NotchPay vers pk_live_..., tests bout en bout Mobile	Clés NotchPay live (pk_live_...) requises.

Phase	Contenu	Dépendances externes
	Money, vérification de signature webhook en conditions réelles.	
Phase 5 — Durcissement	Revue de sécurité complète (checklist section 7, incluant cadrage du chat IA), tests d'IDOR systématiques, mise en place de l'observabilité de sécurité, tests de charge sur les points identifiés en section 8, jobs de rétention/ anonymisation opérationnels (§2.9) et validés.	Aucune.
Phase 6 — Extension anglophone	Activation des classes/filières et banque d'épreuves en anglais, sur un schéma de données déjà prêt (champ langue présent dès la Phase 0).	Approbation préalable du fondateur (cf. brief initial).
Phase 7 — Mode allégé WhatsApp (post-MVP)	Conception détaillée des flux conversationnels (upload photo, affichage quiz, chat contextualisé à l'épreuve, restitution de correction) via WhatsApp Business API, distincts du fonctionnement web. Non chiffrée à ce stade.	Approbation préalable du fondateur ; accès WhatsApp Business API.

Point d'attention pour Claude Code. Respecter les interfaces AIProvider et PaymentProvider dès la phase 0, même en mode mock — c'est ce qui garantit que les phases 3 et 4 se limitent à un changement de configuration et à l'implémentation d'une classe, sans toucher au reste de l'application. Respecter également la règle "une correction par couple élève/épreuve" dès la phase 1 (§4.3) et le cadrage scolaire strict du chat dès son intégration mock (§2.1, §7), pour éviter une refonte de la logique métier lors du passage en mode live.

Le fichier `schema.prisma` (§4, v1.27) est finalisé et prêt à l'emploi dès la Phase 0 — première commande de la Phase 0 : `prisma validate`, puis `prisma migrate dev --name init`.